

SMA Project 3



2026.05.19

PRESENTED BY:

莊雅筑、胡新容、劉芊妤、許鈺偉、陳孝杰、陳柏勳、藍立衡、龍健銘

Content

Week11

Text Embeddings



Week12

BERTopic



第十一周

Text Embeddings



目的

以星宇航空（Starlux）PTT 資料集貫穿全程，從 Word2Vec 到 BERT，執行「找相似文章」與「文章分類」兩個 NLP 任務

核心問題：怎麼把文字變成向量，並用這些向量做下游任務？

任務

1. **Word2Vec**：每個「詞」變成一個向量，可以算詞與詞的相似度、做類比運算、視覺化詞群。
2. **Sentence-BERT**：整段文字壓成一個向量，可以算「句子之間」的語意相似度。
3. **應用**：把上面的 embedding 丟去做「找相似文章」和「Logistic Regression 文章分類」。

資料前處理

```
• # 設定繁體中文詞庫
jieba.set_dictionary('dict.txt')
jieba.load_userdict('user_dict.txt')

# 新增stopwords
with open('stopwords.txt',encoding="utf-8") as f:
    stopWords = [line.strip() for line in f.readlines()]

✓ 0.4s
```

```
# 斷詞函式
def getToken(row):
    seg_list = jieba.lcut(row)
    seg_list = [w for w in seg_list if w not in stopWords and len(w)>1] # 篩選掉停用字與字元

    return seg_list

✓ 0.0s
```

```
# 資料前處理

# 去除一些不需要的欄位
# metaData = origin_data.drop(['artPoster', 'artCategory', 'artComment', 'e_ip', 'insertedDate', 'dataSource'], axis=1)
# 修改後的寫法
metaData = origin_data.drop(
    ['artPoster', 'artCategory', 'artComment', 'e_ip', 'insertedDate', 'dataSource'],
    axis=1,
    errors='ignore'
)
# 只留下中文字
metaData['sentence'] = metaData['artContent'].str.replace(r'\n\n', '。', regex=True)
metaData['sentence'] = metaData['sentence'].str.replace(r'\n', '', regex=True)

metaData['sentence'] = metaData['sentence'].str.split("[, , 。 ! ! [?]? {1,} ")
metaData = metaData.explode('sentence').reset_index(drop=True)

metaData['sentence'] = metaData['sentence'].apply(lambda x: re.sub('[^\u4e00-\u9fff]+', '', x))

metaData['word'] = metaData.sentence.apply(getToken)

metaData = metaData[metaData['word'].apply(len) > 0]

metaData.head(10)

✓ 2.0s
```

為什麼要切成句子？
因為 Word2Vec 訓練時，「一個 list 是一個句子」，**句子越乾淨、上下文窗口才會有意義**

Word2Vec

Bigram 預處理

```
docs = ['new york is united states', 'new york is most populated city in the world', 'i love to stay in new york']

token_ = [doc.split(" ") for doc in docs]
# Phrases 建立bigram,
# 少於min_count的字或bigrams會被忽略,
# 大於threshold的bigrams會被加入
bigram = Phrases(token_, min_count=1, threshold=2)
bigram_phraser = Phraser(bigram)

for sent in token_:
    print(sent) #處理前
    print("=> ", bigram_phraser[sent]) # 處理後，還是原本的句子，只是有抓出的bigram片語會被合併
```

✓ 0.0s

```
['new', 'york', 'is', 'united', 'states']
=> ['new_york', 'is', 'united', 'states']
['new', 'york', 'is', 'most', 'populated', 'city', 'in', 'the', 'world']
=> ['new_york', 'is', 'most', 'populated', 'city', 'in', 'the', 'world']
['i', 'love', 'to', 'stay', 'in', 'new', 'york']
=> ['i', 'love', 'to', 'stay', 'in', 'new_york']
```

```
sents = metaData['word'].to list()
bigrams = Phrases(sents, min_count=1, threshold=1000)
bigram_phrasers = Phraser(bigrams)
metaData['word_list_bigrams'] = list(bigram_phrasers[sents])

metaData.head()
```

✓ 0.0s

- Phrases 會把常一起出現的兩個詞（例如「星宇」+「航空」→「星宇_航空」）合併成一個 token
- threshold 越高、合併越保守

Word2Vec

計算基本資訊

```
word_freq = defaultdict(int)
# 計算詞頻
sents = metaData['word_list_bigrams'].tolist()
for sent in sents: # sent 中的每個句子
    for i in sent: # i 是句子中的每個字
        word_freq[i] += 1

print(f"total unique words in sentences: {len(word_freq)}")
sorted(word_freq, key=word_freq.get, reverse=True)[:10]
```

```
total unique words in sentences: 10195
['星宇', '航空', '公司', '員工', '年終', '飛機', '華航', '長榮', '台中', '罷工']
```

```
print(f"sentence number of corpus: {len(sents)}")
i = 0
for sent in sents:
    i = i + len(sent)
print(f"average length of sentences: {i/len(sents)}")
```

```
sentence number of corpus: 9225
average length of sentences: 4.224932249322493
```

```
# 環境變數設定
%env PYTHONHASHSEED=2025
```

```
env: PYTHONHASHSEED=2025
```

```
# 查看機器的core
cores = multiprocessing.cpu_count()
print(f"number of cores: {cores}")
```

```
number of cores: 16
```

- total unique words : 10195
- sentence number of corpus : 9925
- average length of sentences : 4.22
- number of cores : 16

Word2Vec

建立模型

```
• # 建立模型
w2v_model = Word2Vec(sents,
                      min_count=20, # 小於20次tf的字會被刪除
                      window=2, # 往左右各2的距離
                      vector_size=128, # vector 的維度
                      sample=0.005, # 愈小的話，高tf的字會不容易被選到
                      alpha=0.001, # learning rate
                      min_alpha=0.0005, # 迭代到最小的learning rate，learning rate會慢慢下降至min_
                      negative=0,
                      workers=cores-1, # 用的cpu資源
                      seed=8787,
                      sg = 1, # 0/1 是否使用skip gram
                      epochs= 30,
                      hs=1, # hierarchical softmax
                      )
```

- 詞頻 < 20 直接丟掉
- 上下文窗口左右各 2 個詞
- 詞 向量維度
- 高頻詞下採樣
- 初始 learning rate

- sg=1 (Skip-gram)：相對 CBOW 更適合小資料集 + 低頻詞表現好，**因為PTT 文章量不算超大**
- window=2：代表抓的是「**比較緊密的語法搭配**」而不是寬鬆的主題關聯
- min_count=20：星宇資料集裡只出現過 ≤ 19 次的詞直接砍，**避免雜訊**
- hs=1 + negative=0：用 hierarchical softmax 而不是 negative sampling，**兩個是替代關係**

Word2Vec

結果 - 1 找相似詞

```
• # 檢查最相關的字  
w2v_model.wv.most_similar('飛機',topn=10)
```

✓ 0.0s

```
[('日本', 0.9490016102790833),  
( '知道', 0.9386653900146484),  
( '時間', 0.9385316967964172),  
( '台灣', 0.9335505962371826),  
( '高雄', 0.9276114702224731),  
( '美國', 0.9255407452583313),  
( '國泰', 0.9209871888160706),  
( '歐洲', 0.9192936420440674),  
( '台中', 0.9146592617034912),  
( '成本', 0.9132005572319031)]
```

```
w2v_model.wv.most_similar('年終',topn=10)
```

✓ 0.0s

```
[('股票', 0.9446924328804016),  
( '歐洲', 0.9368185997009277),  
( '成本', 0.9358410239219666),  
( '公司', 0.9219237565994263),  
( '日航', 0.9210205078125),  
( '知道', 0.9189352989196777),  
( '日本', 0.9185351133346558),  
( '引述', 0.9181424379348755),  
( '服務', 0.9159678816795349),  
( '新機', 0.9155805706977844)]
```

Word2Vec

結果 - 2 找多混合詞的鄰居

```
• w2v_model.wv.most_similar(['員工', '年終'], topn=10)
```

✓ 0.0s

```
[('股票', 0.8959171772003174),  
 ('服務', 0.8734714388847351),  
 ('日航', 0.8676159977912903),  
 ('合作', 0.8663333654403687),  
 ('引述', 0.8659377098083496),  
 ('歐洲', 0.8643532991409302),  
 ('原文', 0.858317494392395),  
 ('銘言', 0.8574776649475098),  
 ('成本', 0.8550275564193726),  
 ('成長', 0.8494529128074646)]
```

Word2Vec

結果 - 3 跟這兩個詞最不相關的

```
# 跟兩個字最不相關
```

```
w2v_model.wv.most_similar(negative=['年終', '飛機'], topn=10)
```

✓ 0.0s

```
[('新聞', 0.8306421637535095),  
 ('獎金', 0.7830222249031067),  
 ('餐點', 0.7075164318084717),  
 ('有錢', 0.6444366574287415),  
 ('財報', 0.6375221014022827),  
 ('網友', 0.5508993864059448),  
 ('裡面', 0.5113587975502014),  
 ('旅客', 0.44615066051483154),  
 ('太少', 0.41591063141822815),  
 ('原來', 0.3793383240699768)]
```

Word2Vec

結果 - 4 兩個詞的關係、相似度

```
# 計算兩個字之間的關係
```

```
w2v_model.wv.similarity("年終","長榮")
```

✓ 0.0s

```
0.66144097
```

```
w2v_model.wv.similarity("星宇","飛機")
```

✓ 0.0s

```
0.90445924
```

Word2Vec

結果 - 5 找出邊緣詞(比較字詞之間，最不相關的)

```
# 比較字詞間，誰最不相關 (邊緣)
```

```
w2v_model.wv.doesnt_match(["旅客", "星宇", '飛機'])
```

✓ 0.0s

'旅客'

結果 - 6 類比運算、相對關係

```
# 相對關係
```

```
w2v_model.wv.most_similar(positive=["年終"], negative=["星宇"], topn=5)
```

✓ 0.0s

```
[('員工', 0.5330454707145691),  
( '共體時艱', 0.39684513211250305),  
( '認為', 0.3885459303855896),  
( '反正', 0.37329360842704773),  
( '薪資', 0.3723338842391968)]
```


Word2Vec

降維並視覺化

降維：利用PCA tSNE

```
def reduceDim(mat,method:str='PCA',dim:str=2,perplexity = 25,learning_rate = 400):
```

```
    method_dict = {  
        "PCA":PCA(n_components=dim,iterated_power = 1000,random_state=0),  
        "TSNE":TSNE(n_components=dim,random_state=0,perplexity=perplexity,learning_rate=learning_rate),  
    }  
    new_feat = method_dict[method].fit_transform(mat)
```

```
    return new_feat
```

拿到list of words 的vector

```
def getVecs(model,words:list):  
    vecs = []  
    for i in words:  
        vecs.append(model.wv[i])  
    return np.vstack(vecs)
```

擴展相似的字詞

```
def expandPosWord(model, words:list, top_n:int, split = True):
```

```
    if split == False:  
        wp = model.wv.most_similar(words,topn = top_n)  
        return wp  
    expand = []
```

```
    for w in words:  
        wp = model.wv.most_similar(w,topn = top_n)  
        for i in wp:  
            expand.append(i[0])
```

```
    return list(set(expand))
```

```
expandPosWord(w2v_model,['年終','飛機'],top_n = 10)
```

```
['日航',  
'服務',  
'成本',  
'日本',  
'知道',  
'時間',  
'歐洲',  
'股票',  
'台灣',  
'公司',  
'美國',  
'國泰',  
'台中',  
'引述',  
'高雄',
```

- PCA：線性降維、快、保留全域結構，適合先看大概形狀
- t-SNE：非線性、慢、保留局部鄰居關係，適合看「分群長什麼樣」

擴展相似詞

Word2Vec

降維並視覺化

二維散狀圖

```
# 畫出兩維的散佈圖
def plotScatter(vec_df):
    """
    vec_df: 字詞及其兩個維度的值
    """
    plt.figure(figsize=(15,15))
    fontManager.addfont('./TaipeiSansTCBeta-Regular.ttf')
    plt.rcParams['font.sans-serif'] = ['Taipei Sans TC Beta']
    plt.rcParams['font.size'] = '16'

    p = sns.scatterplot(x="dim1", y="dim2",
                       data=vec_df)
    for line in range(0, vec_df.shape[0]):
        p.text(vec_df["dim1"][line],
              vec_df["dim2"][line],
              ' ' + vec_df["word"][line].title(),
              horizontalalignment='left',
              verticalalignment='bottom', size='medium',
              weight='normal'
              ).set_size(15)
    plt.show()

# 畫出三維的散佈圖
def plotScatter3D(vec_df):
    vec_df['size'] = .5
    if 'color' not in vec_df.columns:
        vec_df['color'] = 'blue'
    fig = px.scatter_3d(
        vec_df, 'dim1', 'dim2', 'dim3', text='word', width=800, height=800, color='color', size='size'
    )
    fig.show()
```



Word2Vec

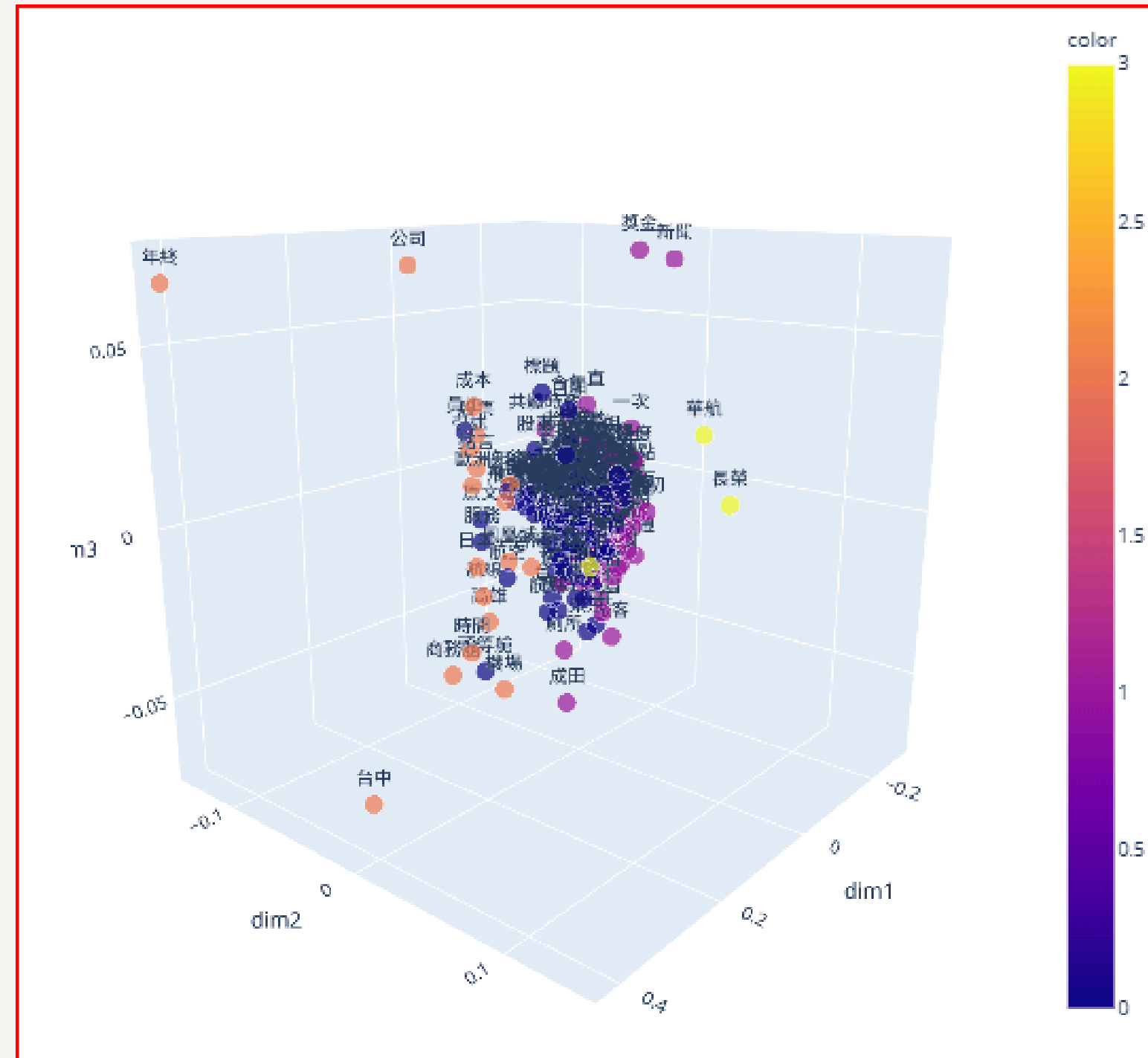
降維並視覺化 將字分群(kmeans)

3D 散狀圖(三維)

```
# kmeans分群
from sklearn.cluster import KMeans
from sklearn_extra.cluster import KMedoids
# 只使用word vector 去分群
def cluster(X,method = 'kmeans',n = 2):

    method_dict = {
        'kmeans':KMeans(n_clusters=n, random_state=0),
        'kmedos':KMedoids(n_clusters=n, random_state=0)
    }
    method_dict[method].fit(X)
    result = method_dict[method].predict(X)
    return result
```

```
new_feat = reduceDim(feats, method='PCA', dim = 20)
d3_feat = reduceDim(feats, method='PCA', dim = 3)
word_df = pd.DataFrame({
    "word": sample_words,
    "color": cluster(new_feat, n=4),
    "dim1": d3_feat[:, 0],
    "dim2": d3_feat[:, 1],
    "dim3": d3_feat[:, 2],
})
plotScatter3D(word_df)
```



- 分群在較高維空間做，結果比較穩定
- 如果直接在 3D 上分群，會被降維時的失真嚴重影響

Transformers Embeddings

以BERT模型為範例

```
# 中文 bert-base-chinese
bert_ch = SentenceTransformer('google-bert/bert-base-chinese')

bert_ch.tokenizer.add_special_tokens({'pad_token': '[PAD]'})

# 範例句子
sentences = [
    "星宇航空的服務真的很棒",
    "這家航空公司的飛機餐很好吃",
    "我不喜歡這次的飛行體驗"
]

# 使用 encode() 對資料做embedding
embeddings_ch = bert_ch.encode(sentences)

# Compute cosine-similarities
cosine_scores = util.cos_sim(embeddings_ch, embeddings_ch)

# 印出句子間的cosine similarity分數
result_df = get_result_df(sentences, cosine_scores)
result_df
```

✓ 4.8s

	sentence1	sentence2	score
0	星宇航空的服務真的很棒	這家航空公司的飛機餐很好吃	0.837406
2	這家航空公司的飛機餐很好吃	我不喜歡這次的飛行體驗	0.786230
1	星宇航空的服務真的很棒	我不喜歡這次的飛行體驗	0.785202

- 0.83 (服務很棒 vs 飛機餐很好吃)：這兩句皆為「正面評價」，且主題皆與航空服務高度相關，因此在空間中距離最近，分數最高。
- 0.78 (正面讚美 vs 負面體驗)：雖然語義上一個是讚美、一個是不喜歡，但由於兩者討論的客體都是「航空體驗」，在基礎模型未經細節微調的情況下，仍會給予極高的相似度評價。

因為上面課堂中用的模型，結果出來「正評」跟「負評」的分數會太過接近 (0.78-0.83)，因此換另一個模型

`model_name = 'paraphrase-multilingual-MiniLM-L12-v2'`

Transformers Embeddings

換模型 paraphrase-multilingual-MiniLM-L12-v2

```
model_name = 'paraphrase-multilingual-MiniLM-L12-v2'
bert_ch = SentenceTransformer(model_name)

# 範例句子
sentences = [
    "星宇航空的服務真的很棒",
    "這家航空公司的飛機餐很好吃",
    "我不喜歡這次的飛行體驗"
]

# 使用 encode() 對資料做embedding
embeddings_ch = bert_ch.encode(sentences)

# Compute cosine-similarities
cosine_scores = util.cos_sim(embeddings_ch, embeddings_ch)

# 印出句子間的cosine similarity分數
result_df = get_result_df(sentences, cosine_scores)
result_df
```

✓ 13m 11.2s

modules.json: 100% 229/229 [00:00<00:00, 36.8kB/s]

- **分數高 (0.54) = 語境與情緒接近**

意義：模型認為這兩句話在向量空間中的「方向」比較一致，具備較強的關聯性

兩者皆為正面評價且都圍繞著航空體驗

- **分數中 (0.41) = 主題相關但情緒不同**

意義：模型察覺到情緒上的拉伸（情緒位移），因此分數明顯比純粹正面的組合還要低。

存在一正一負的情緒差異

- **分數低 (0.30) = 語義衝突最明顯**

意義：這兩句話在向量空間中的距離最遠，代表模型成功識別出它們是語義衝突的對立面。

	sentence1	sentence2	score
0	星宇航空的服務真的很棒	這家航空公司的飛機餐很好吃	0.541189
2	這家航空公司的飛機餐很好吃	我不喜歡這次的飛行體驗	0.410714
1	星宇航空的服務真的很棒	我不喜歡這次的飛行體驗	0.302536

使用embedding做NLP任務

任務一：找相似文章

```
def find_similar_articles(query_idx, top_k=5):
    """輸入文章 index，顯示前 top_k 篇最相似文章"""
    query_emb = corpus_embeddings[query_idx]
    cos_scores = util.cos_sim(query_emb, corpus_embeddings)[0]
    top_results = torch.topk(cos_scores, k=top_k + 1) # +1 排除自身

    title_q = df_sim['artTitle'].iloc[query_idx]
    board_q = df_sim['board'].iloc[query_idx]
    print(f"Query [{index={query_idx}}] [{board_q}]: [{title_q}] ")
    print("=" * 70)

    count = 0
    for score, idx in zip(top_results[0], top_results[1]):
        if idx.item() == query_idx:
            continue
        title = df_sim['artTitle'].iloc[idx.item()]
        board = df_sim['board'].iloc[idx.item()]
        print(f" Top {count+1}. [{board}] [{title}] 相似度: {score.item():.4f}")
        count += 1
        if count >= top_k:
            break
    print()

# 以三篇不同的文章示範
for qidx in [0, 50, 100]:
    find_similar_articles(qidx, top_k=5)
```

✓ 0.0s

```
Query (index=0) [Aviation]: [[問題] 星宇 華航 新加坡 航空選擇]
=====
Top 1. [Aviation] [[問題] 星宇 華航 新加坡 航空選擇] 相似度: 0.8358
Top 2. [Aviation] [[問題] 星宇 華航 新加坡 航空選擇] 相似度: 0.8249
Top 3. [Aviation] [[問題] 星宇 華航 新加坡 航空選擇] 相似度: 0.8175
Top 4. [Aviation] [[問題] 星宇 華航 新加坡 航空選擇] 相似度: 0.7958
Top 5. [Aviation] [[問題] 星宇 華航 新加坡 航空選擇] 相似度: 0.7790

Query (index=50) [Aviation]: [[閒聊] 星宇航空找金城武拍廣告可以洗白嗎?]
=====
Top 1. [Aviation] [[閒聊] 星宇航空找金城武拍廣告可以洗白嗎?] 相似度: 0.9654
Top 2. [Aviation] [[閒聊] 星宇航空找金城武拍廣告可以洗白嗎?] 相似度: 0.9580
Top 3. [Aviation] [[閒聊] 星宇航空找金城武拍廣告可以洗白嗎?] 相似度: 0.9516
Top 4. [Aviation] [[閒聊] 星宇航空找金城武拍廣告可以洗白嗎?] 相似度: 0.9394
Top 5. [Aviation] [[閒聊] 星宇航空找金城武拍廣告可以洗白嗎?] 相似度: 0.9218

Query (index=100) [Aviation]: [Re: [情報] 星宇航空增租A321neo/減租A330neo]
=====
Top 1. [Aviation] [Re: [情報] 星宇航空增租A321neo/減租A330neo] 相似度: 0.8263
Top 2. [Aviation] [Re: [情報] 星宇航空增租A321neo/減租A330neo] 相似度: 0.8214
Top 3. [Aviation] [Re: [情報] 星宇航空增租A321neo/減租A330neo] 相似度: 0.8161
Top 4. [Aviation] [Re: [情報] 星宇航空增租A321neo/減租A330neo] 相似度: 0.8121
Top 5. [Aviation] [Re: [情報] 星宇航空增租A321neo/減租A330neo] 相似度: 0.7882
```

- 對整個文集一次 encode 成矩陣 `corpus\_embeddings`
- 拿其中一篇當 query，算它跟全部文章的 cosine 相似度
- `torch.topk` 抓前 k 大

使用embedding做NLP任務

任務一：找相似文章

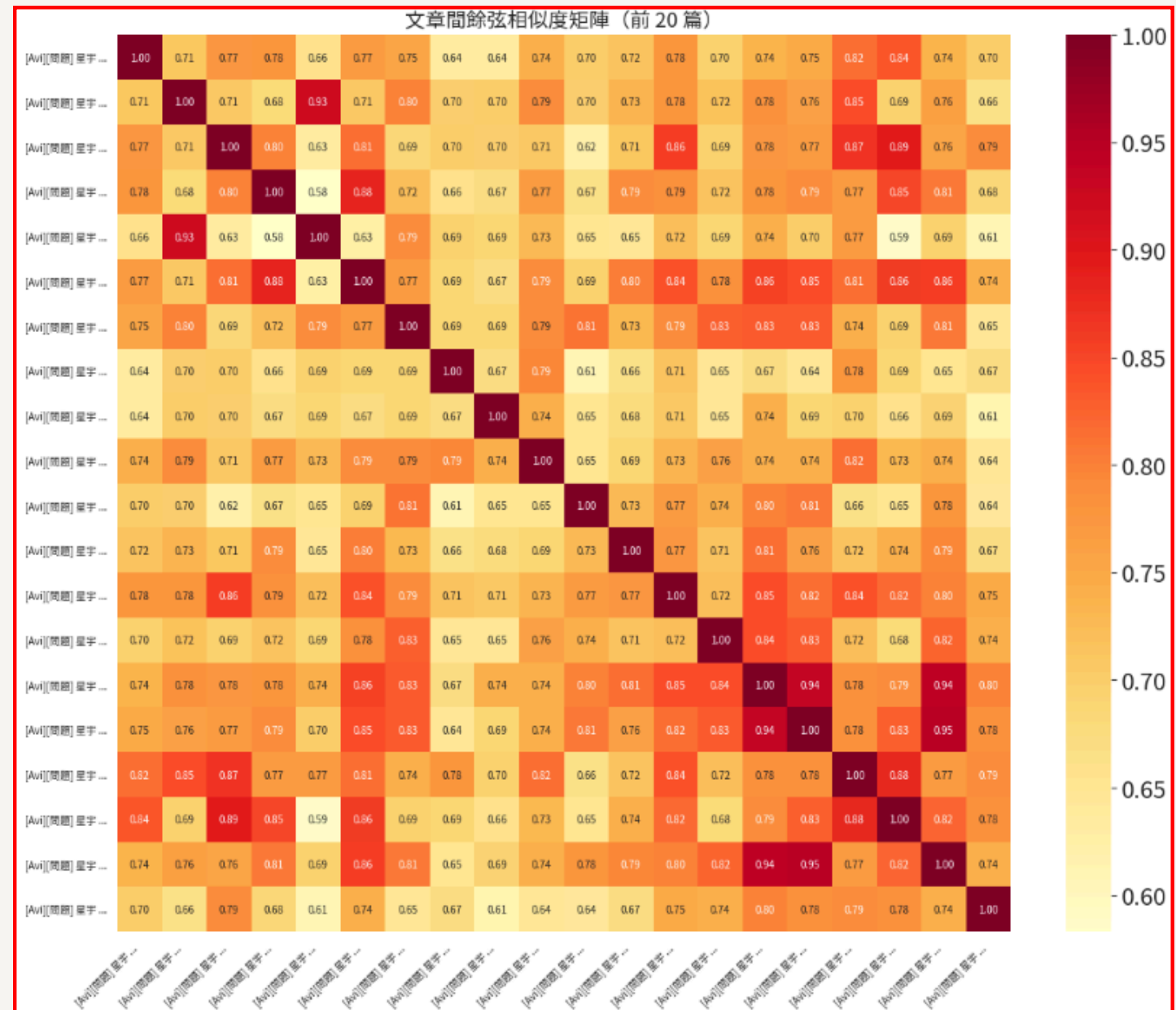
```
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.font_manager import fontManager

fontManager.addfont('./TaipeiSansTCBeta-Regular.ttf')
plt.rcParams['font.sans-serif'] = ['Taipei Sans TC Beta']

# 取前 20 篇文章，計算兩兩相似度並畫 heatmap
sample_idx = list(range(20))
sample_labels = [
    f"[{df_sim['board'].iloc[i][:3]},{df_sim['artTitle'].iloc[i][:8]}]..."
    for i in sample_idx
]
sample_embs = corpus_embeddings[sample_idx]
sim_matrix = util.cos_sim(sample_embs, sample_embs).cpu().numpy()

plt.figure(figsize=(12, 10))
sns.heatmap(
    sim_matrix, annot=True, fmt='.2f', cmap='YlOrRd',
    xticklabels=sample_labels, yticklabels=sample_labels,
    annot_kws={'size': 7}
)
plt.title('文章間餘弦相似度矩陣 (前 20 篇)', fontsize=14)
plt.xticks(rotation=45, ha='right', fontsize=7)
plt.yticks(rotation=0, fontsize=7)
plt.tight_layout()
plt.show()
```

✓ 0.9s



- 改用 `paraphrase-multilingual-MiniLM-L12-v2`，相似度更乾淨。
- 用 heatmap 把前 20 篇文章的兩兩相似度矩陣畫出來，可以直接看出哪些文章彼此最像。

使用embedding做NLP任務

任務二：文章分類

以文章 embedding 作為特徵，訓練 **Logistic Regression** 分類器，預測文章屬於 **Aviation**（航空板）還是 **Stock**（股票板）。

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# 用 embedding 作為特徵，board 作為標籤
X = corpus_embeddings.cpu().numpy()
y = df_sim['board']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
print(f"訓練集: {len(X_train)} 筆 | 測試集: {len(X_test)} 筆\n")

# 訓練 Logistic Regression
clf = LogisticRegression(max_iter=1000, random_state=42)
clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)

print("=== Logistic Regression 分類結果 ===\n")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}\n")
print(classification_report(y_test, y_pred))
```

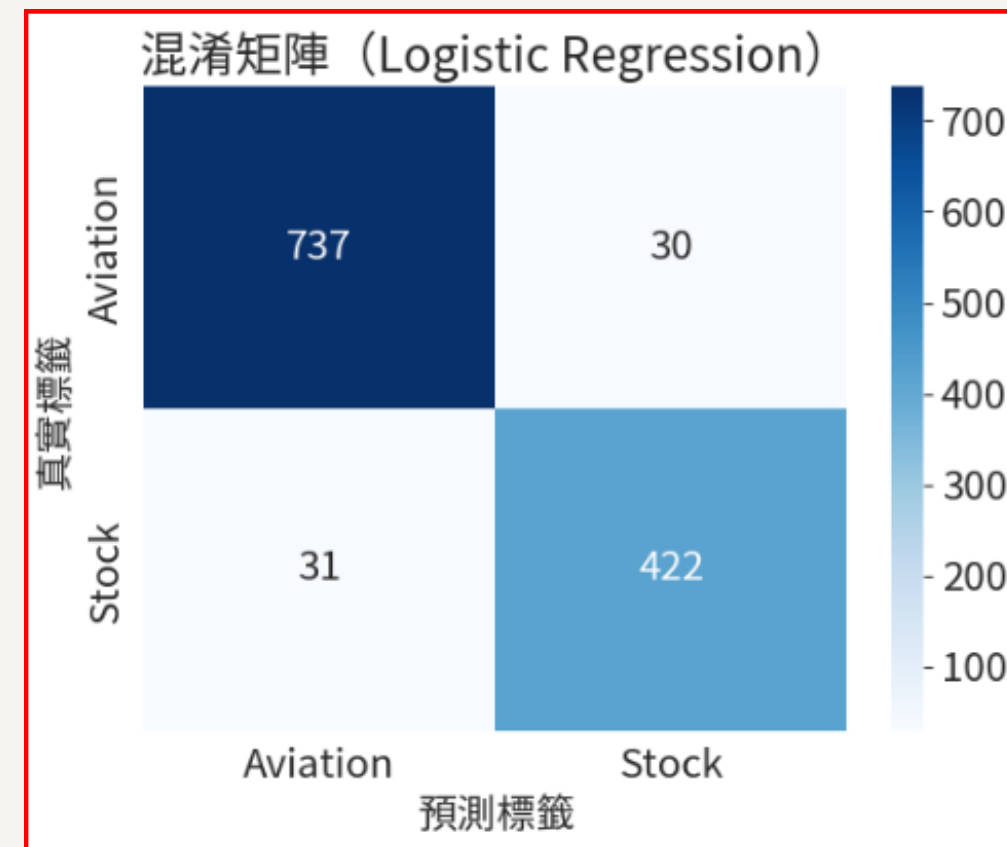
訓練集：2846 筆 | 測試集：1220 筆

=== Logistic Regression 分類結果 ===

Accuracy: 0.9500

	precision	recall	f1-score	support
Aviation	0.96	0.96	0.96	767
Stock	0.93	0.93	0.93	453
accuracy			0.95	1220
macro avg	0.95	0.95	0.95	1220
weighted avg	0.95	0.95	0.95	1220

- 特徵工程：BERT embedding（768 維）當特徵向量。
- 標籤：`board` 欄位（Aviation / Stock）。
- 切分：7:3，後半段用 `stratify=y` 保證類別比例。
- 模型：Logistic Regression（線性分類器，因為 embedding 已經把非線性吃掉了，下游用線性就夠）。
- 評估：accuracy + precision / recall / f1 + 混淆矩陣 heatmap。



第十二周

BERTopic



BERT

Token Classification



資料前處理

星宇航空資料集

- 使用開源ptt-crawler爬蟲，抓取PTT 文章。
- 版：「Aviation」、「Stock」
- 標題關鍵字：「星宇」
- 時間: 2025-12-01 ~ 2026-03-18

Step 1 — 資料載入與欄位整理

- 移除不必要欄位
- artDate 轉換為日期格式
- 移除 artContent 為空值的筆數

Step 2 — 文字清理

- 移除網址(http, www)
- 統一換行符號 (`\n` → `\n`)，移除 `\r`, `\t`
- 同義詞替換("台灣人"→"台灣"、"中華航空"→"華航")
- 移除空格與特殊符號(全形符號、括號、箭頭等)

Step 3 — 斷句

- 依標點符號斷句 (。！？；，等)
- 過濾條件：長度 > 1 且包含至少一個中文字

Step 4 — 展開為每句一列

- 每篇文章的句子列表 explode 為獨立的列
- 移除斷句後產生的空字串

比較 ↘

Dataset

總句數：10370
總文章數：4008
日期範圍：2025-12-06 ~ 2026-03-17
句子平均長度：12.1 字
句子最短／最長：2 ／ 340 字

BERT BASE

BERT TINY

中研院所做的實驗數據

參數量	102M	12M
Perplexity	1.88	8.07
WS(F1)	97.60%	96.98%
POS (ACC)	95.67%	95.11%
NER (F1)	81.18%	74.21%
花費時間	10:24	1:00

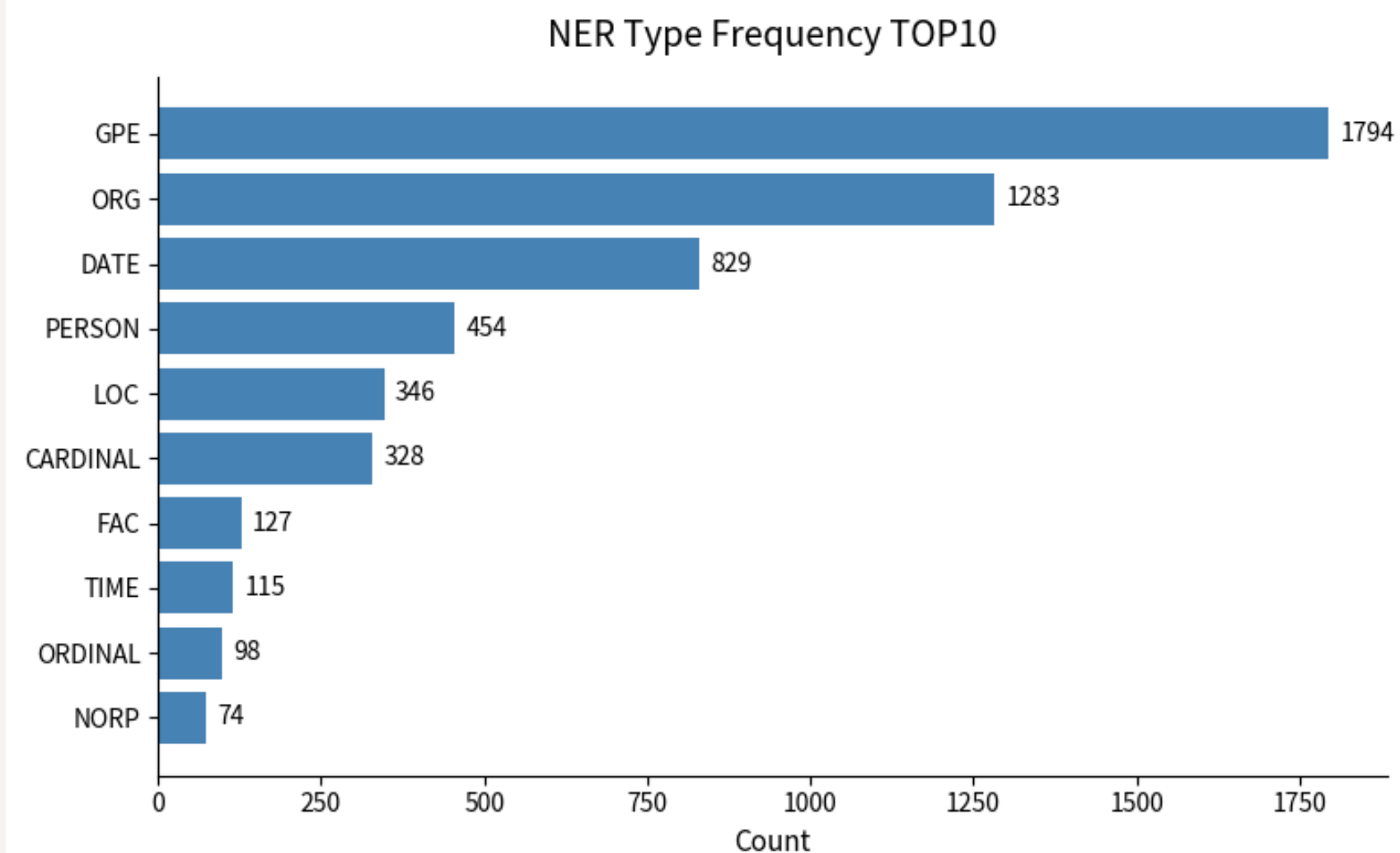
數值越低越好，越不感到困惑

星宇資料集

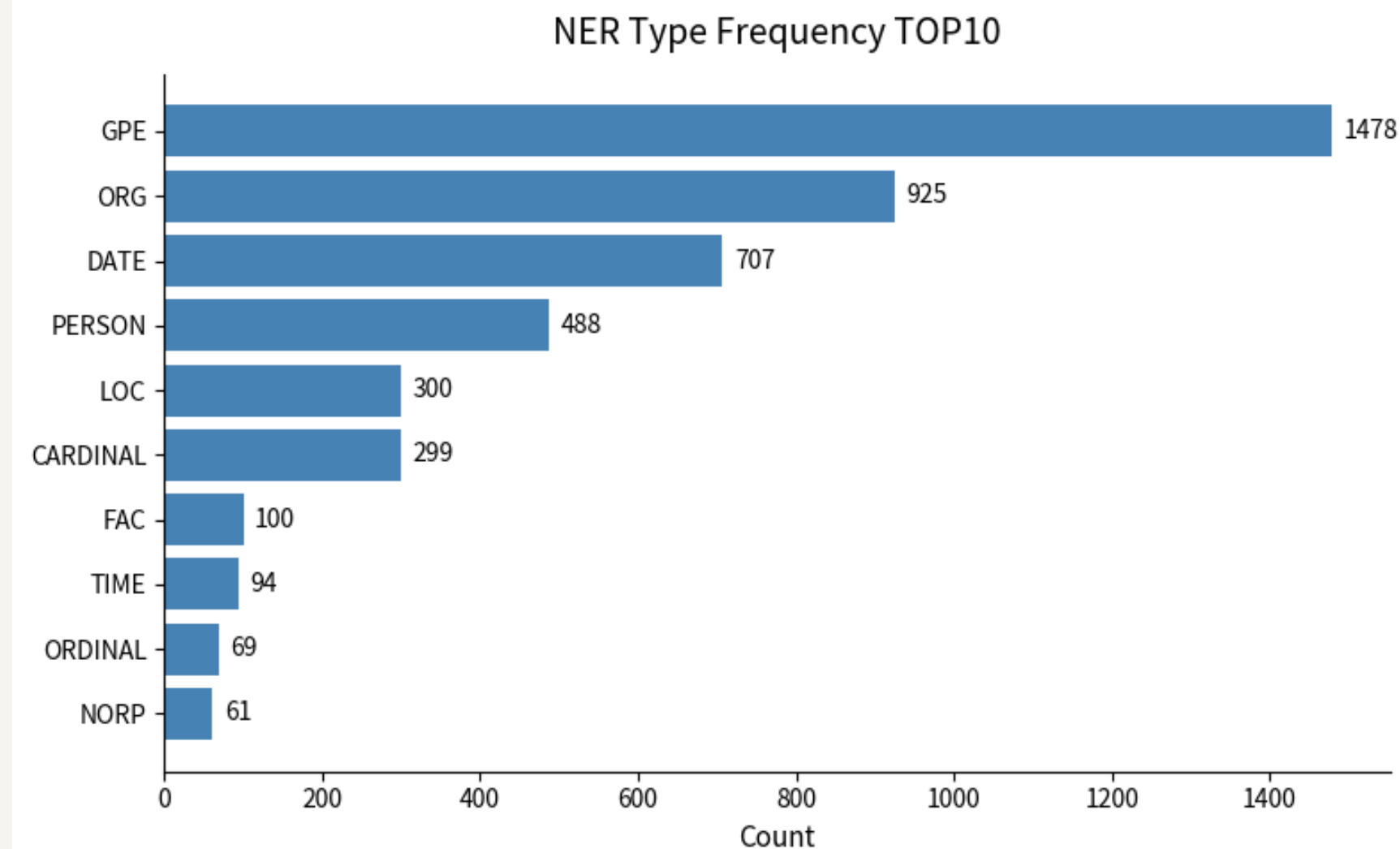
NER Type ↘

BASE辨識出來的效果整體比TINY還要好

BERT Base



BERT TINY

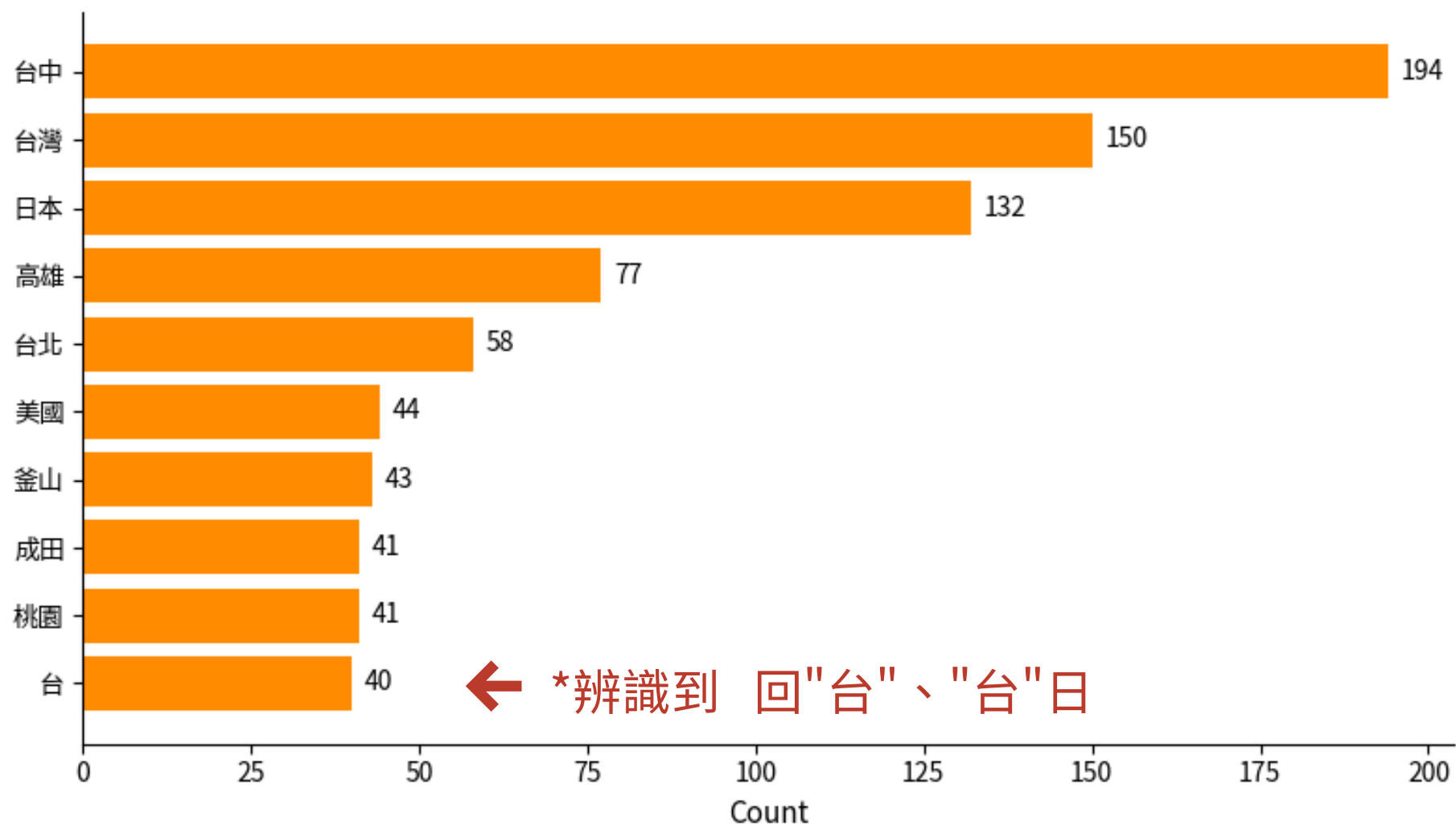


地理位置 ↘

星宇進駐台中，並且布局日本、美國、韓國航線

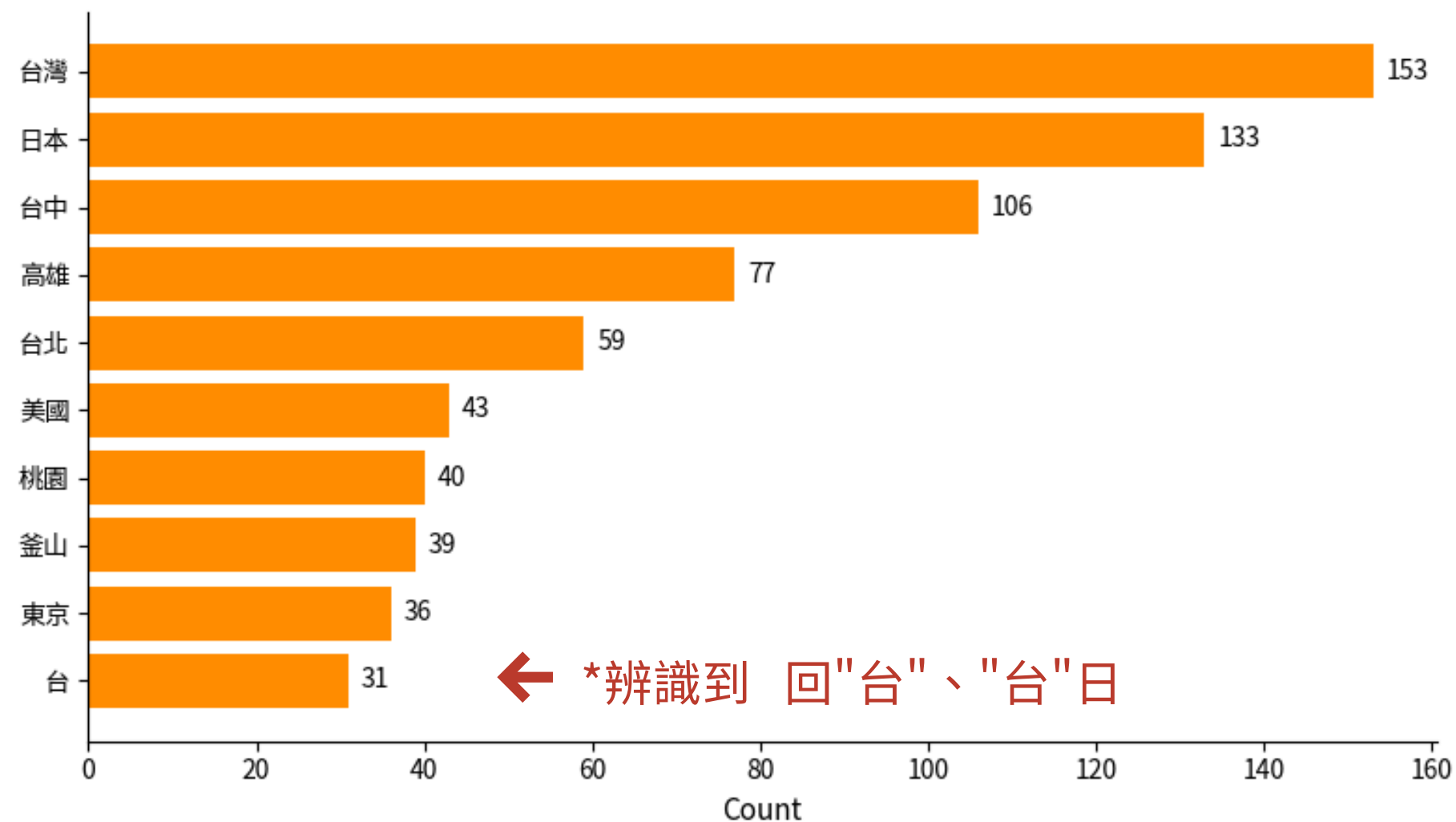
BERT Base

GPE (地理位置) TOP10



BERT TINY

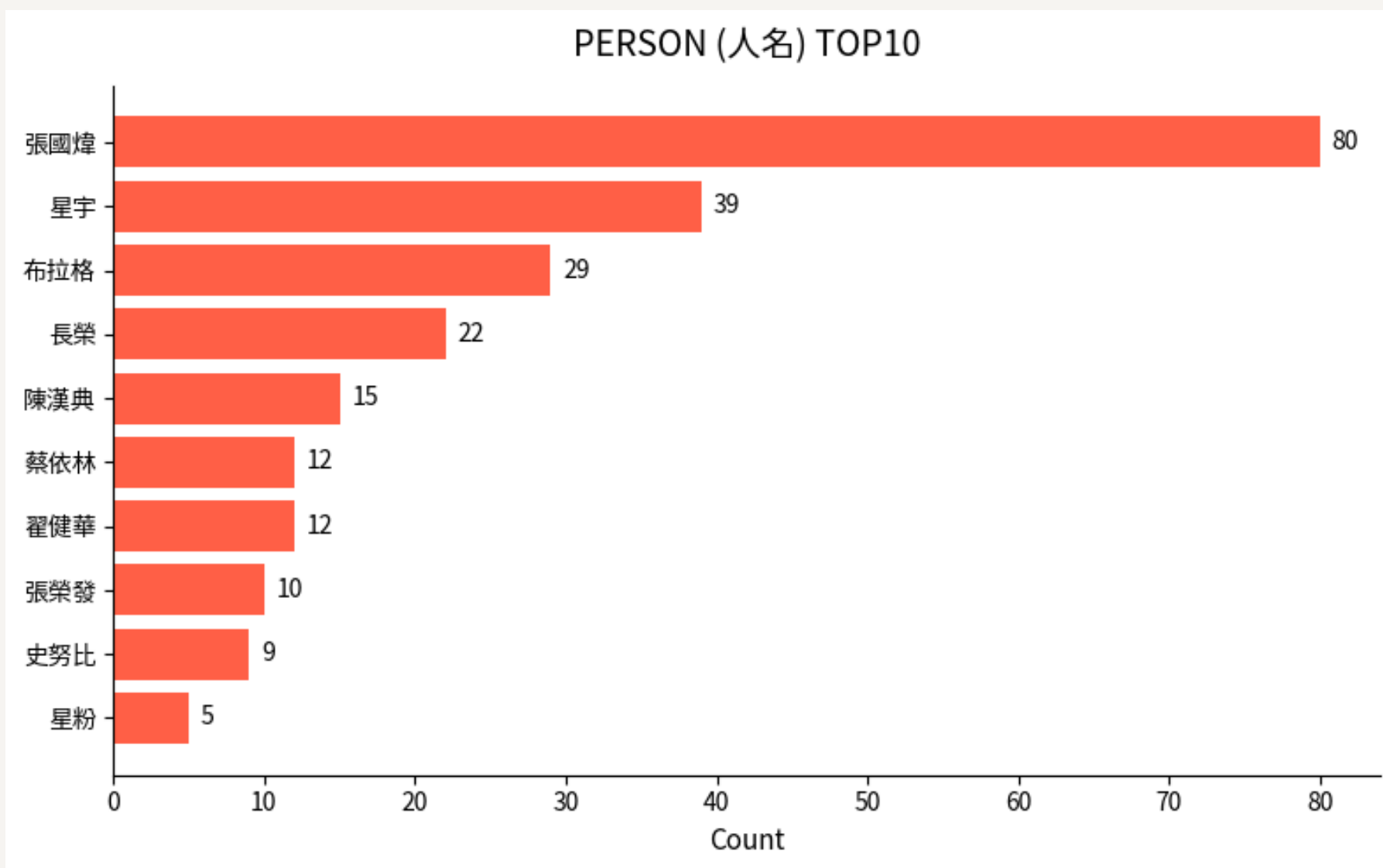
GPE (地理位置) TOP10



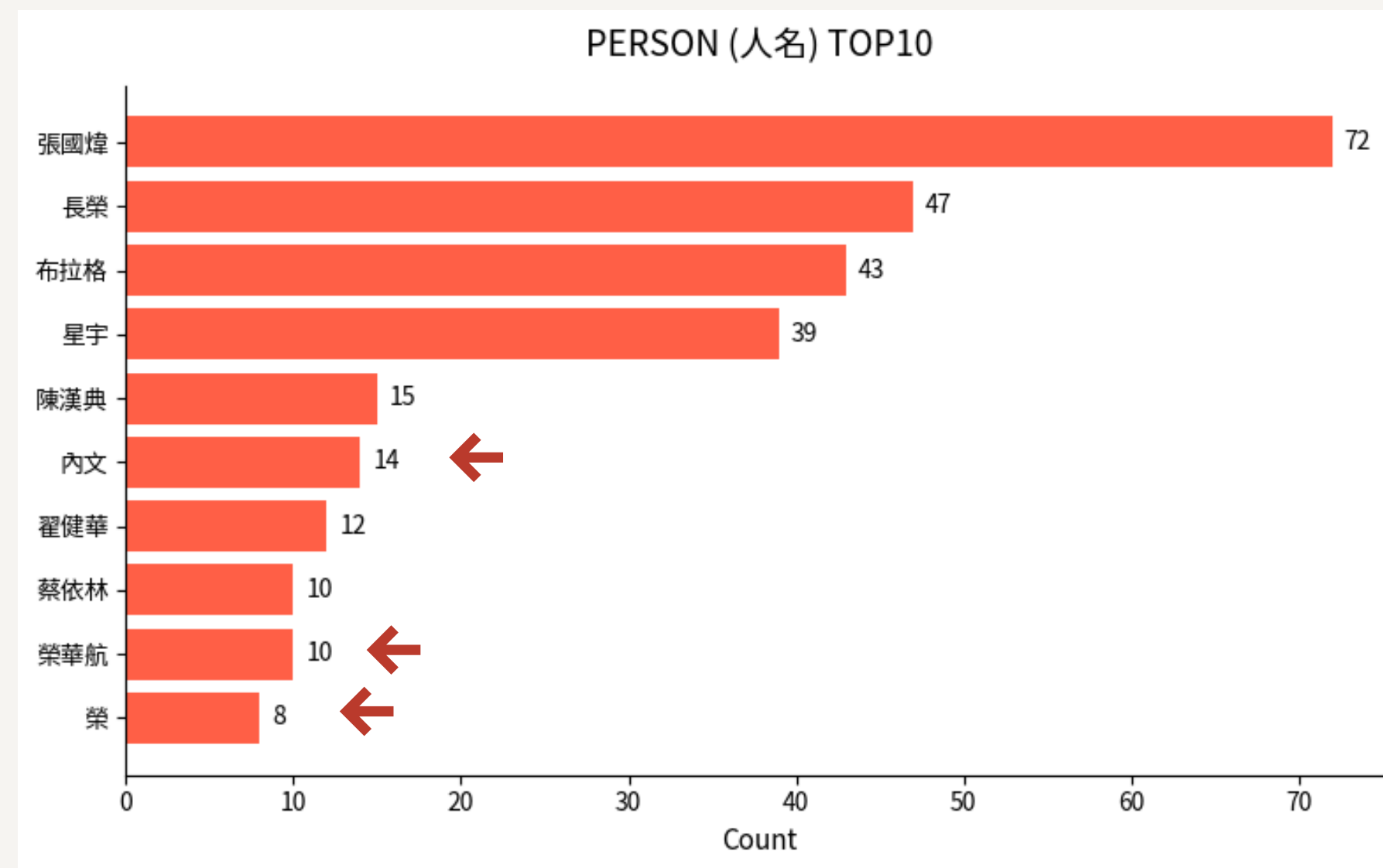
NER人名 ↘

TINY針對"長榮華航"，"內文"進行NER時錯誤率較高

BERT Base



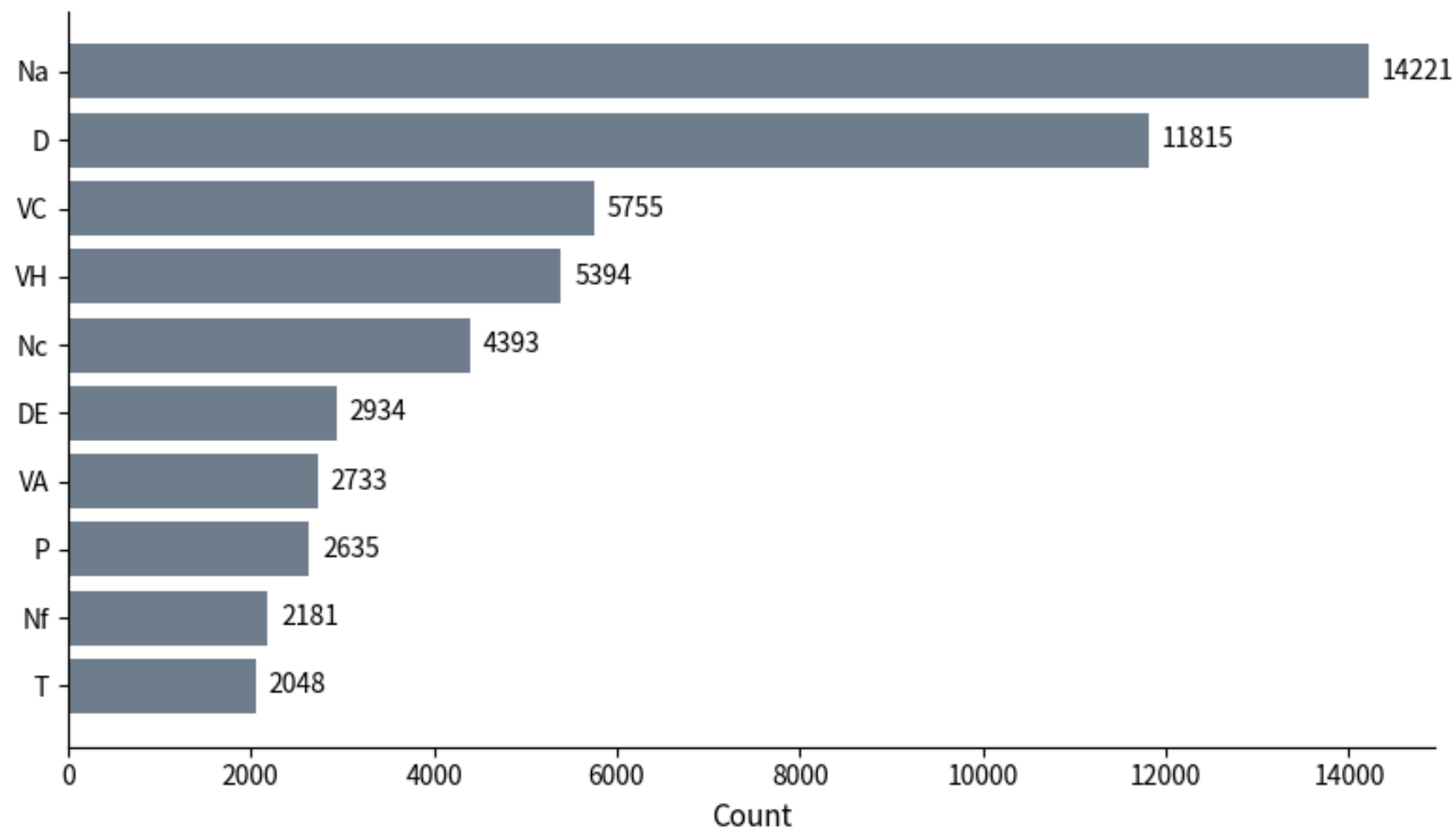
BERT TINY



標詞性 ↘

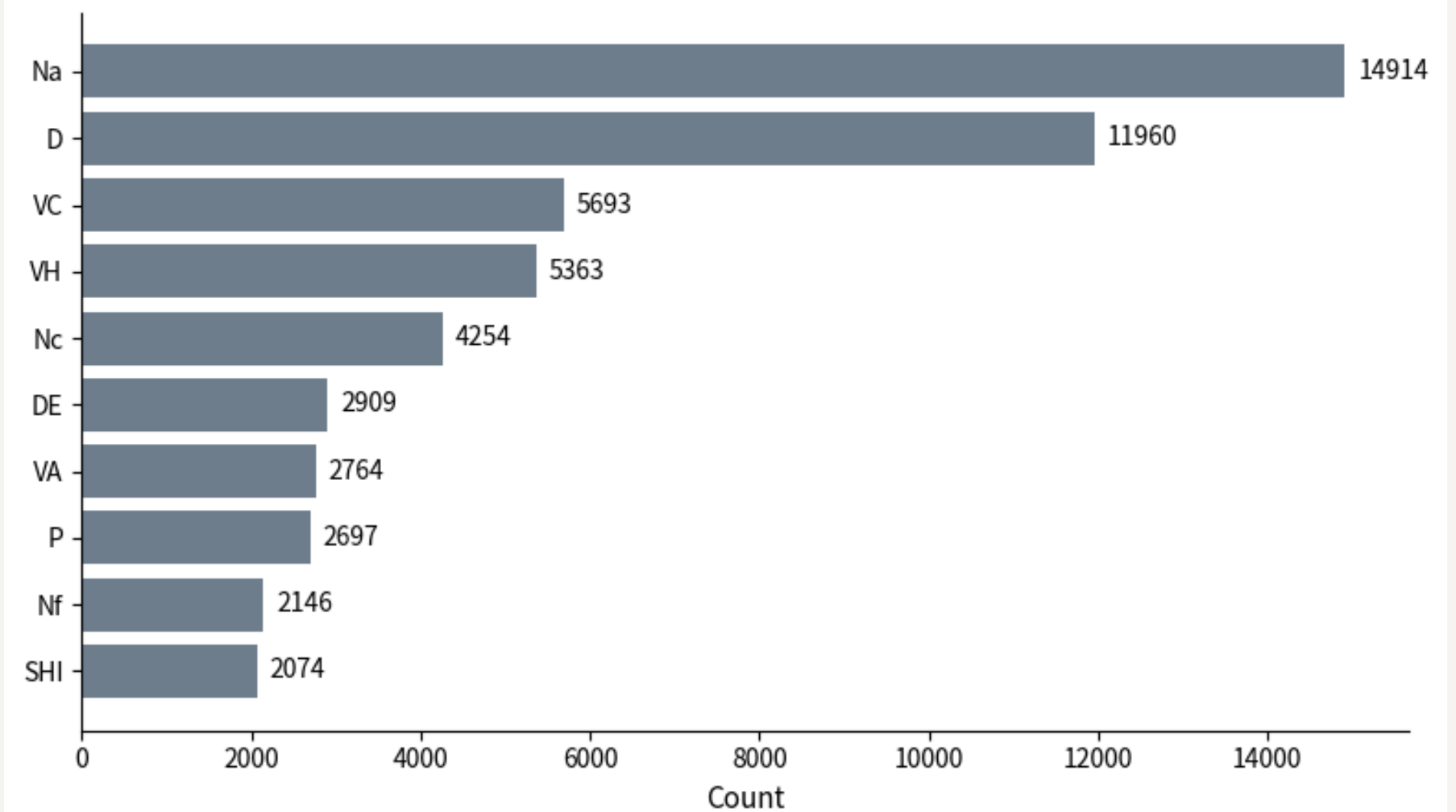
BERT Base

POS Tag Frequency TOP10



BERT TINY

POS Tag Frequency TOP10

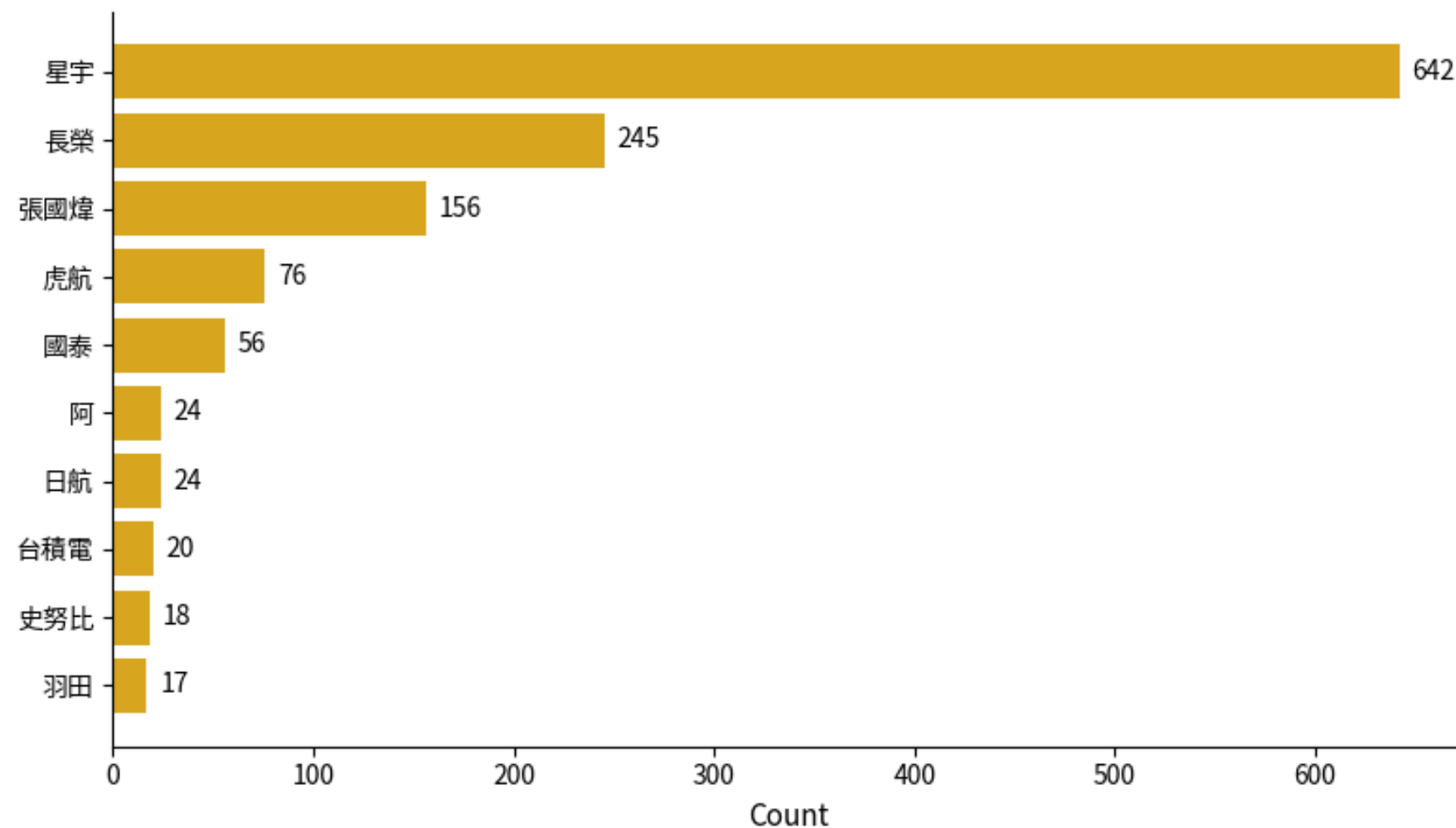


專有名詞 ↘

BASE辨識出來的比TINY還要多

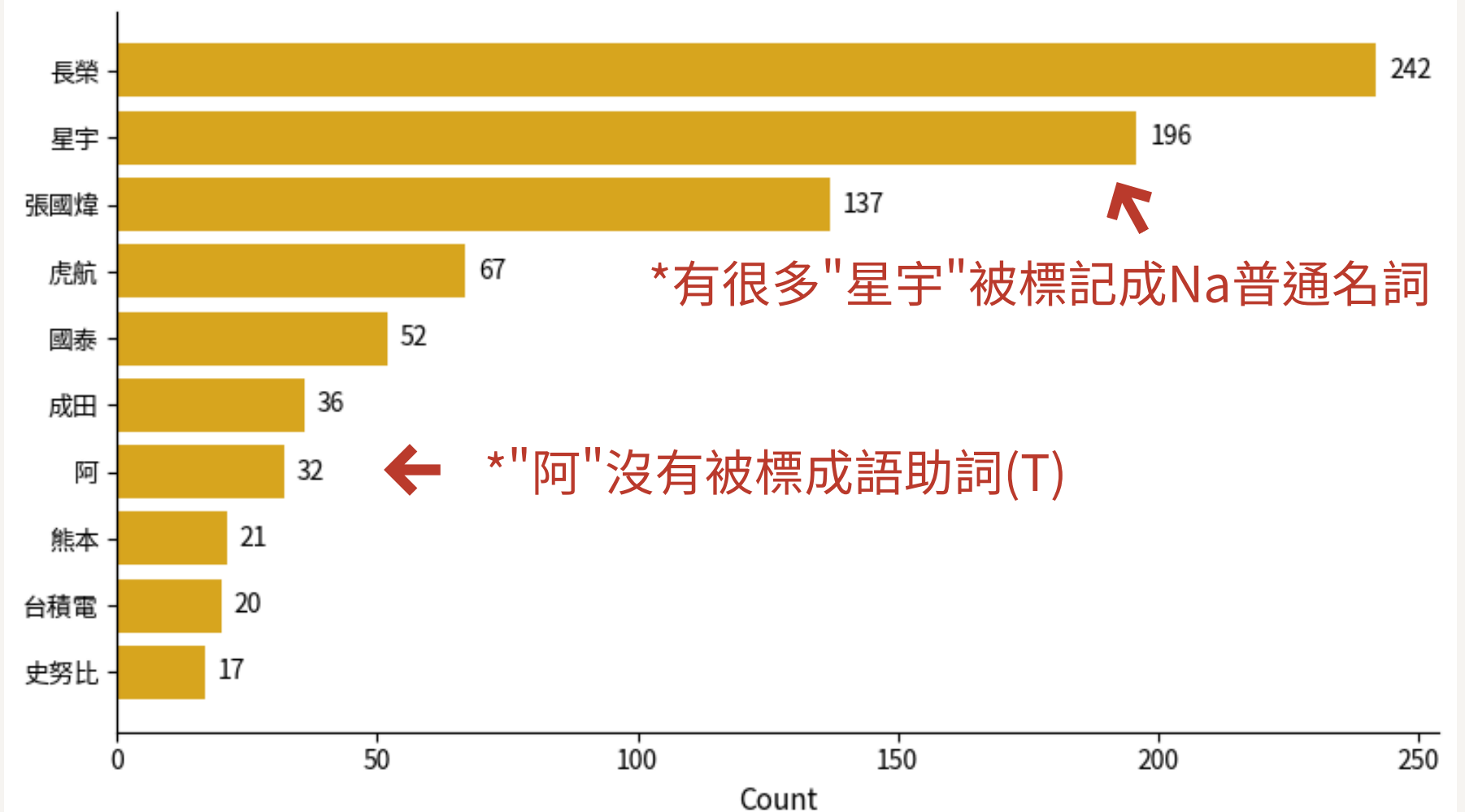
BERT Base

Noun / Nb (專有名詞) TOP10



BERT TINY

Noun / Nb (專有名詞) TOP10



專有名詞-阿

BERT會把語助詞(T)「阿」
辨識成Nb

ALBERT沒有這問題

ALBERT Tiny

這(Nep) 家(Nf) 怎麼(D) 常常(D) 有(V_2) 新聞(Na) 阿(T)

這家怎麼常常有新聞阿

ALBERT Base

這(Nep) 家(Nc) 怎麼(D) 常常(D) 有(V_2) 新聞(Na) 阿(T)

這家怎麼常常有新聞阿

BERT Tiny

這(Nep) 家(Nf) 怎麼(D) 常常(D) 有(V_2) 新聞(Na) 阿(Nb)

這家怎麼常常有新聞阿

BERT Base

這(Nep) 家(Nc) 怎麼(D) 常常(D) 有(V_2) 新聞(Na) 阿(Nb)

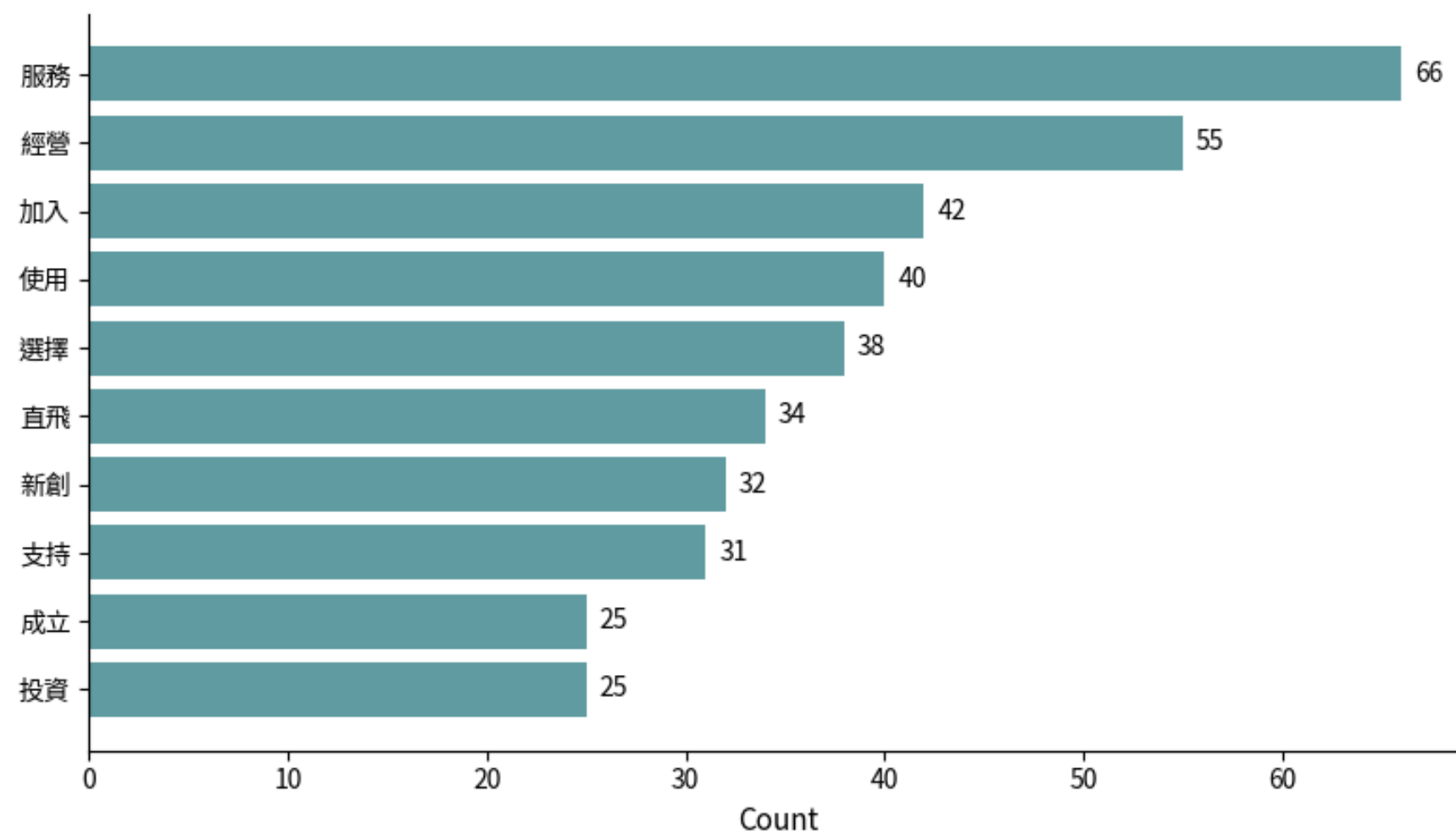
這家怎麼常常有新聞阿

及物動詞 ↘

「服務」提及率很高，大家對航空業的服務品質尤其看中

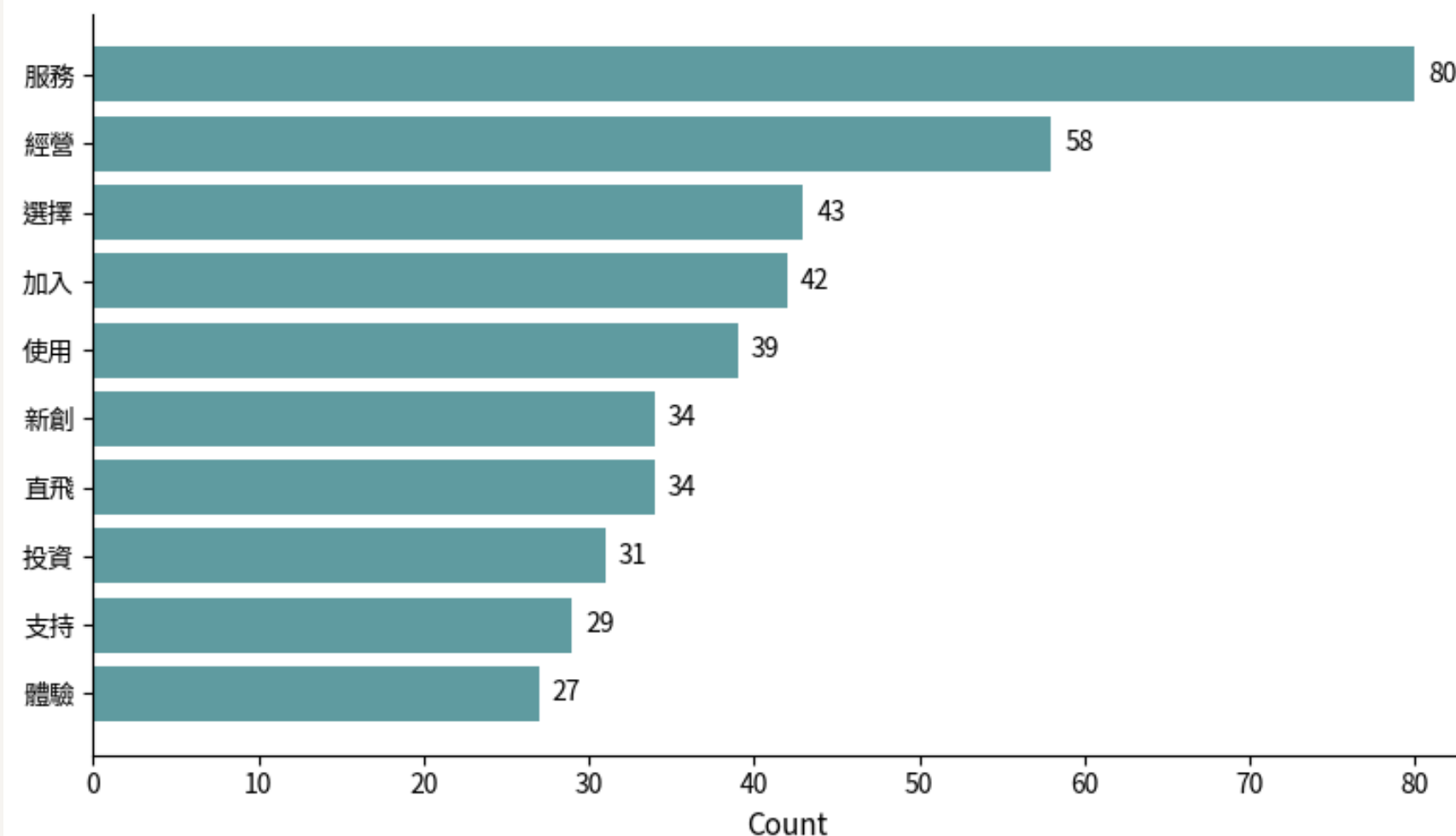
BERT Base

VC (及物動詞) TOP10



BERT TINY

VC (及物動詞) TOP10



BERT

Sequence Classification

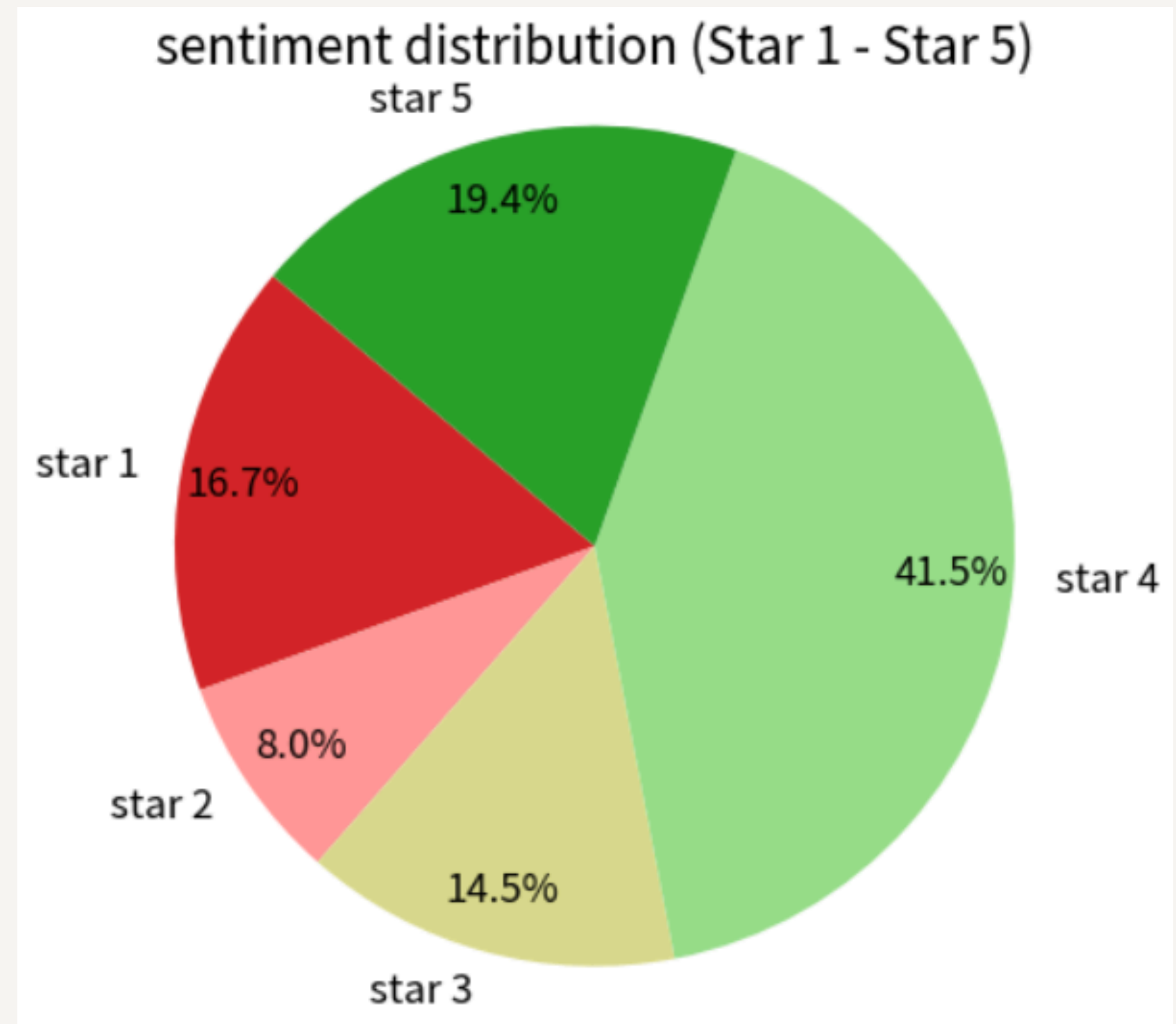


Sentiment Classification

Model：助教使用的
techthiyanes/chinese_sentiment

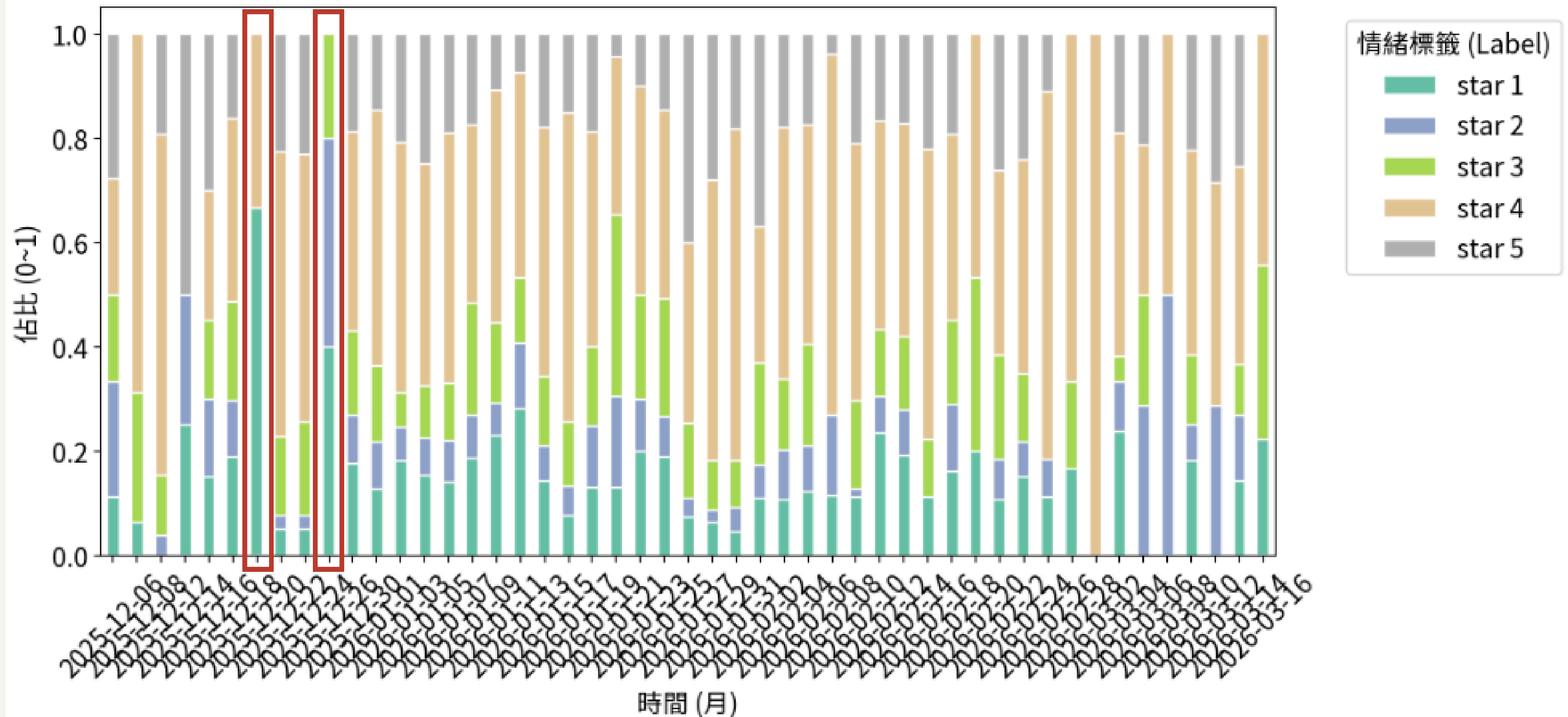
情緒(start 1到star 5)：

1. Semi-negation
2. Negation
3. Neutral
4. Semi-positive
5. Positive

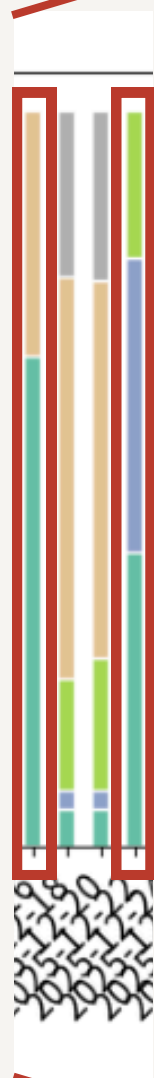


Sentiment Classification - 兩天為一單位

各情緒標籤的佔比時間趨勢



Sentiment Classification



航空消息
全部
起手官方
操作安全

BERT

Text Clustering



BERTopic 結果

	Topic	Count	Name	Representation
0	-1	5797	-1_飛機_星宇_長榮	[, 飛機, 星宇, 長榮, 公司, 日本, 航空, 華航, 票價, 台灣, 員工, 年終, 航線, 精品, 高雄, 錢, 成田, 機場, 航空業, 航點, 航空公司, 賣, 前, 桃園, 便宜, 疫情, 董, 機票, 吃, 旅客]
1	0	958	0_員工_公司_股票_股東	[員工, 公司, 股票, 股東, 老闆, 跳槽, 賺錢, 新創, 星宇, 錢, 股價, , 星粉, 資方, 企業, 反串, 講話, 年終, 時艱, 賭, 選擇權, 共體, 虧, 工作, 堆, 賠錢, 罷工, 文章, 勞工, 努力]
2	1	303	1_釜山_首爾_布拉格_韓國	[釜山, 首爾, 布拉格, 韓國, 蘇黎世, 日本, 轉機, 芬蘭, 歐洲, 曼谷, 瑞士, 東京, 航點, 仁川, 台灣, 沖繩, 機場, 捷克, 神戶, 出發, 熊本, 東南亞, 桃園, 赫爾辛基, 便宜, 郵輪, 旅行團, 波蘭, 高鐵, 歐積]
3	2	188	2_頭等艙_艙_商務_經濟艙	[頭等艙, 艙, 商務, 經濟艙, 廁所, 席, 豪華, 制止, 乘客, 優先, 專用, 客戶, 頭等, 座位, 旅客, 艙席, 布簾, 用餐, 四艙, 前排, 漢典, 豪經艙, 陳, 提供, 睡衣, 機上, 洗手間, 過夜, 排隊, 坐著]

公司營運與員工待遇

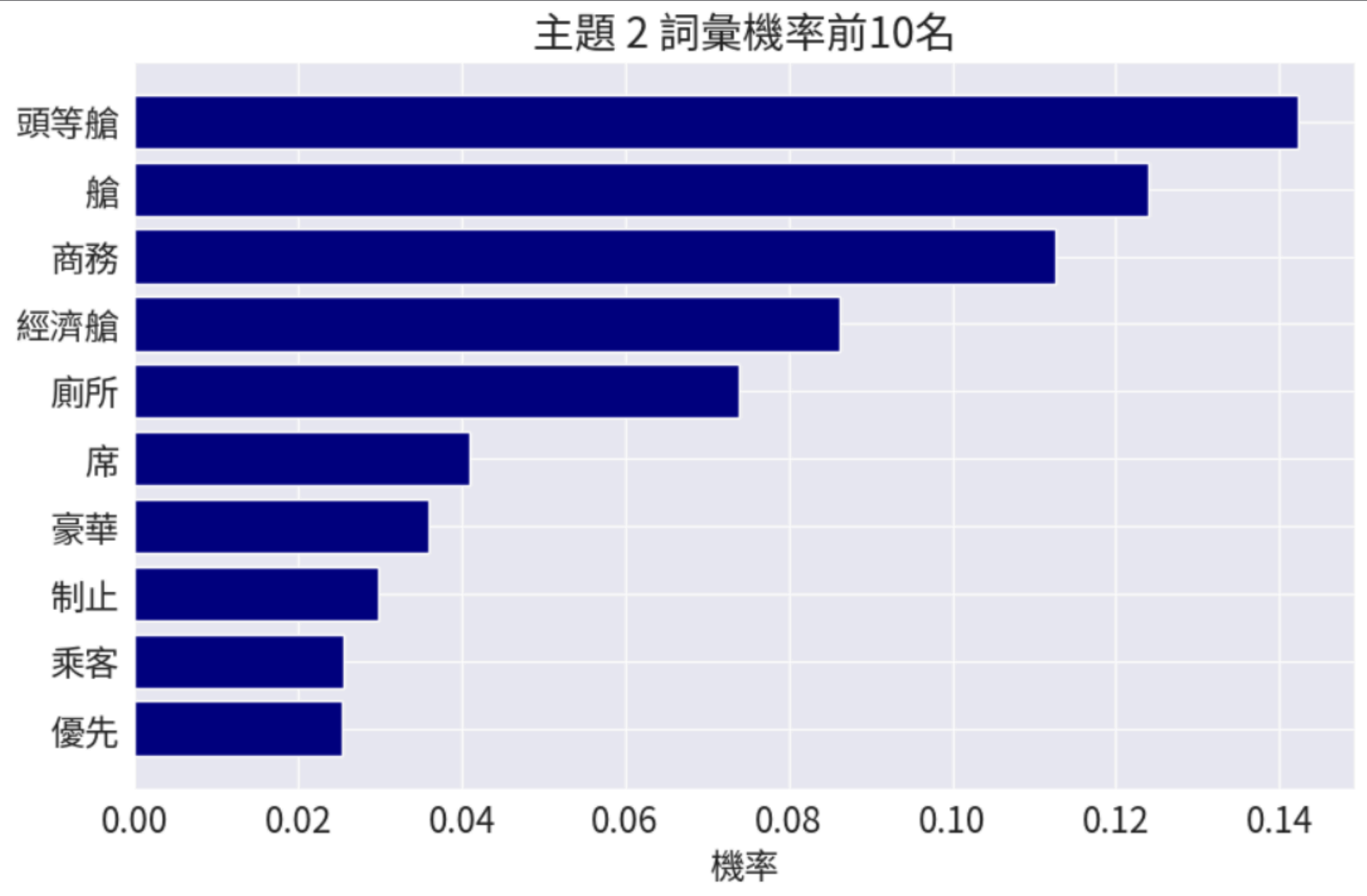
航線與旅遊目的地

艙等與機上服務

BERTopic 結果

Topic	Count	Name	Representation
0	-1	5797 -1_飛機_星宇_長榮	[, 飛機, 星宇, 長榮, 公司, 日本, 航空, 華航, 票價, 台灣, 員工, 年終, 航線, 精品, 高雄, 錢, 成田, 機場, 航空業, 航點, 航空公司, 賣, 前, 桃園, 便宜, 疫情, 董, 機票, 吃, 旅客]
1	0	958 0_員工_公司_股票_股東	[員工, 公司, 股票, 股東, 老闆, 跳槽, 賺錢, 新創, 星宇, 錢, 股價, , 星粉, 資方, 企業, 反串, 講話, 年終, 時艱, 賭, 選擇權, 共體, 虧, 工作, 堆, 賠錢, 罷工, 文章, 勞工, 努力]
2	1	303 1_釜山_首爾_布拉格_韓國	[釜山, 首爾, 布拉格, 韓國, 蘇黎世, 日本, 轉機, 芬蘭, 歐洲, 曼谷, 瑞士, 東京, 航點, 仁川, 台灣, 沖繩, 機場, 捷克, 神戶, 出發, 熊本, 東南亞, 桃園, 赫爾辛基, 便宜, 郵輪, 旅行團, 波蘭, 高鐵, 歐積]
3	2	188 2_頭等艙_艙_商務_經濟艙	[頭等艙, 艙, 商務, 經濟艙, 廁所, 席, 豪華, 制止, 乘客, 優先, 專用, 客戶, 頭等, 座位, 旅客, 艙席, 布簾, 用餐, 四艙, 前排, 漢典, 豪經艙, 陳, 提供, 睡衣, 機上, 洗手間, 過夜, 排隊, 坐著]

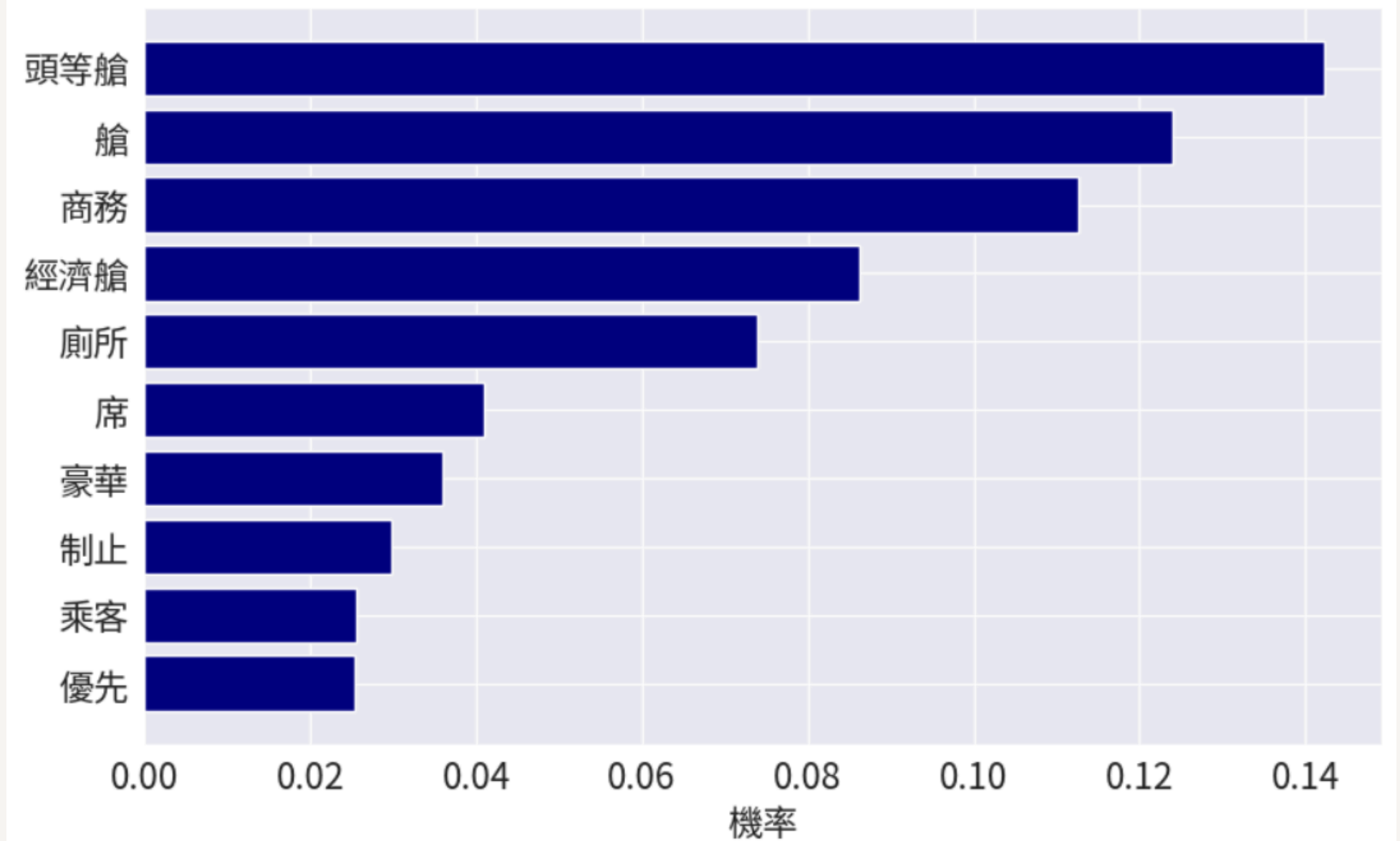
以Topic 2 為例 詞彙機率前10名



BERTopic 結果

Topic	Count	Name	Representation
0	-1	5797 -1_飛機_星宇_長榮	[, 飛機, 星宇, 長榮, 公司, 日本, 航空, 華航, 票價, 台灣, 員工, 年終, 航線, 精品, 高雄, 錢, 成田, 機場, 航空業, 航點, 航空公司, 賣, 前, 桃園, 便宜, 疫情, 董, 機票, 吃, 旅客]
1	0	958 0_員工_公司_股票_股東	[員工, 公司, 股票, 股東, 老闆, 跳槽, 賺錢, 新創, 星宇, 錢, 股價, , 星粉, 資方, 企業, 反串, 講話, 年終, 時艱, 賭, 選擇權, 共體, 虧, 工作, 堆, 賠錢, 罷工, 文章, 勞工, 努力]
2	1	303 1_釜山_首爾_布拉格_韓國	[釜山, 首爾, 布拉格, 韓國, 蘇黎世, 日本, 轉機, 芬蘭, 歐洲, 曼谷, 瑞士, 東京, 航點, 仁川, 台灣, 沖繩, 機場, 捷克, 神戶, 出發, 熊本, 東南亞, 桃園, 赫爾辛基, 便宜, 郵輪, 旅行團, 波蘭, 高鐵, 歐積]
3	2	188 2_頭等_艙_艙_商務_經濟艙	[頭等艙, 艙, 商務, 經濟艙, 廁所, 席, 豪華, 制止, 乘客, 優先, 專用, 客戶, 頭等, 座位, 旅客, 艙席, 布簾, 用餐, 四艙, 前排, 漢典, 豪經艙, 陳, 提供, 睡衣, 機上, 洗手間, 過夜, 排隊, 坐著]

主題 2 詞彙機率前10名

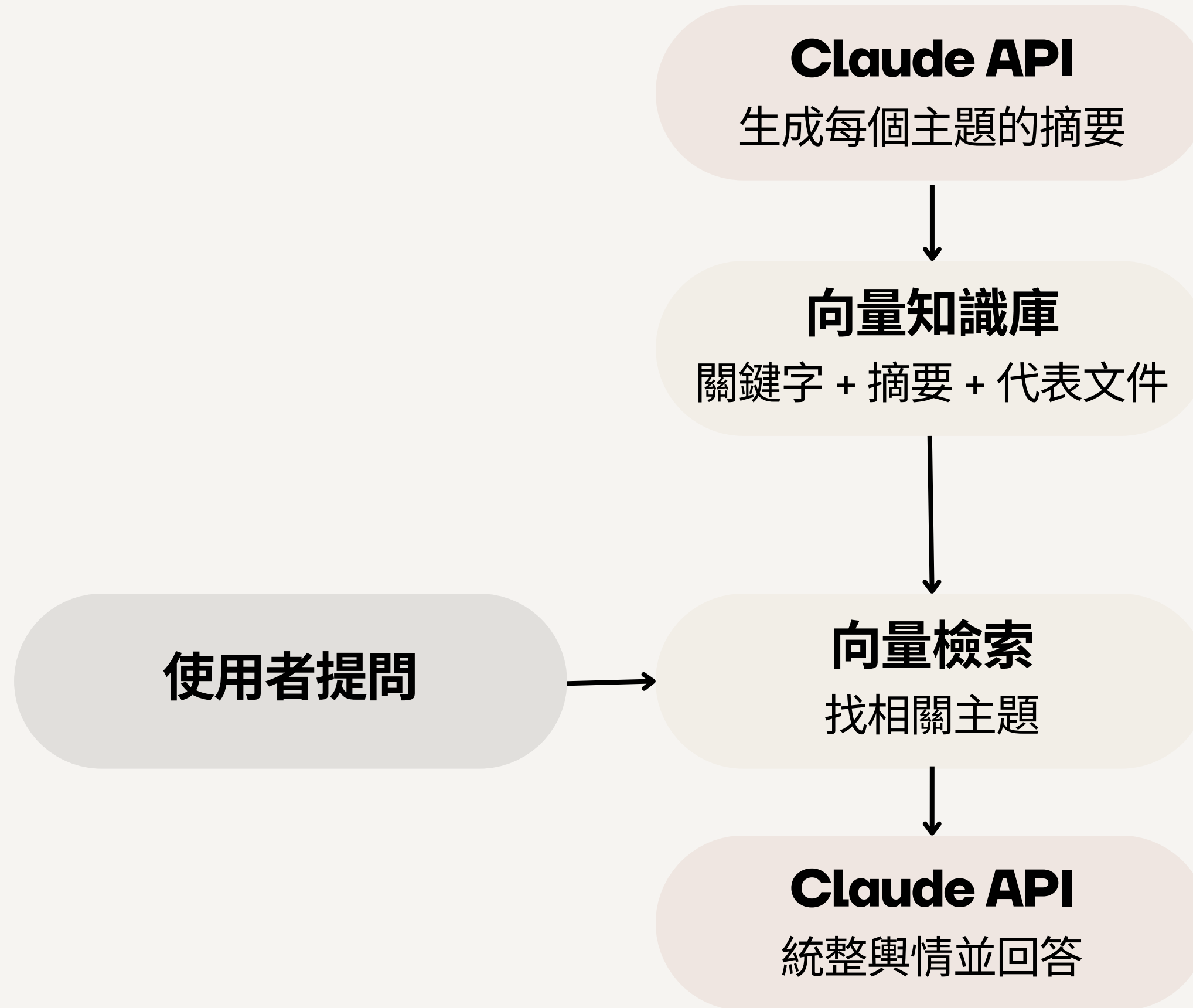


BERT

(延伸) RAG



流程



主題摘要

```
for topic_id in range(min(10, len(zh_topic_model.get_topic_info()) - 1)):
    rep_docs = zh_topic_model.get_representative_docs(topic_id)
    keywords = zh_topic_model.get_topic(topic_id)[:5]
    keyword_str = "、".join([w for w, _ in keywords])

    prompt = f"""
        以下是網友討論星宇航空的留言，關鍵詞為：{keyword_str}

        留言內容：
        {chr(10).join(rep_docs)}

        請用2~3句話摘要這群留言在討論什麼主題，以及網友的主要觀點。
        """
```

主題 0

關鍵字：員工、股票、公司、股東、賺錢

摘要：網友討論星宇航空的獲利能力與股東權益議題。主要觀點認為星宇航空的股票很難賺錢，即使不是股東身份，公司仍應該為股東創造獲利。整體反映出對星宇航空營運獲利表現的質疑態度。

主題 1

關鍵字：韓國、釜山、日本、首爾、便宜

摘要：這群留言主要討論**芬蘭航空的航線調整策略**，以及**韓國與日本二三線城市航點的營運效益比較**。

網友認為芬蘭航空因曼谷航線載客率不佳而延伸至墨爾本以增加轉機客源，但對開拓台灣航線興趣缺缺。在韓國航點方面，網友普遍認為除了首爾和釜山具競爭力外，其他韓國城市的利潤遠不如日本二三線城市，因此航空公司會優先布局日本市場。

主題 2

關鍵字：頭等艙、艙、商務、經濟艙、席

摘要：這則留言在討論**不同艙等乘客使用廁所的權限問題**。

網友提出疑問：如果商務艙沒有客人時，經濟艙乘客是否能使用商務艙廁所？同時指出即使是商務艙乘客也不能使用頭等艙的廁所，質疑這種艙等間的使用限制規定。整體反映出網友對於艙等隔離政策的好奇與討論。

主題 3

關鍵字：精、學人、加油、維修、根

摘要：根據留言內容，網友們重複使用「學人精」一詞來討論星宇航空。「學人精」在台語中帶有貶義，意指愛模仿他人或裝模作樣的人。從網友重複強調這個詞彙來看，他們似乎對星宇航空某些模仿其他航空公司的做法或營運方式感到不滿或嘲諷。

主題 4

關鍵字：不演、帥、太、奇怪、可怕

摘要：根據這則留言，網友在討論星宇航空的某個事件或情況，主要觀點是認為星宇航空「不演了」，暗示該公司不再掩飾或偽裝某些做法。網友透過重複強調「不演了」一詞，表達對星宇航空某種行為或態度轉變的關注，可能帶有諷刺或批評的意味。

向量知識庫

```
knowledge_base = []

for topic_id, summary in topic_summaries.items():
    keywords = zh_topic_model.get_topic(topic_id)[:5]
    keyword_str = "\ ".join([w for w, _ in keywords])
    rep_docs = zh_topic_model.get_representative_docs(topic_id)

    knowledge_base.append({
        "topic_id": topic_id,
        "keywords": keyword_str,
        "summary": summary,
        "rep_docs": rep_docs,
        "search_text": f"{keyword_str} {summary}"
    })

# 轉成向量
search_texts = [item["search_text"] for item in knowledge_base]
kb_embeddings = bert_sentence_model.encode(search_texts, show_progress_bar=True)
```

使用者問答

```
answer = rag_query("網友對星宇航空的餐飲有什麼看法？")  
print(answer)
```

```
# 網友對星宇航空餐飲的看法分析
```

```
## 主要觀點
```

根據提供的資料，關於星宇航空餐飲的討論**相對有限但正面**，主要集中在一位網友的詳細分享。

```
## 正面意見 🍌
```

從代表留言中可以看出明確的正面評價：

```
### 1. **餐點豐富度高**
```

- 提供完整的用餐體驗，從開胃前菜到主餐都有詳細規劃
- 前菜種類多樣化：包括厚揚豆腐、煙燻鮭魚、蘿蔔捲、山藥秋葵、海帶沙拉等日式小菜
- 主餐選用高級食材：胡同日本和牛臂肉上蓋牛排

```
### 2. **特調飲品備受讚賞**
```

- 網友特別強調對「橙香迷霧」的喜愛
- 使用龍眼酒調製，香氣迷人
- 評價「越喝越順口」，顯示口感層次佳

```
### 3. **整體服務水準**
```

- 餐飲品質符合或超越期待
- 細節處理用心（如日式料理呈現）

```
## 反面意見 🚩
```

從提供的資料中未發現針對餐飲的明確負面評價。

```
## 分析總結
```

雖然資料中僅有一則詳細的餐飲體驗分享，但該網友的評價**相當正面且具體**，特別突出星宇航空在餐飲品質和創意調酒方面的優勢。這種詳細的正面分享通常代表深刻的滿意體驗。不過需要注意的是，樣本數量有限，無法全面代表所有乘客的觀點。

使用者問答

```
answer = rag_query("網友對星宇航空開拓新航線有什麼看法？")
print(answer)
```

```
## 一、主要觀點整理

根據提供的討論區資料，網友對星宇航空開拓新航線的看法呈現**分歧態度**，整體討論較為零散，但可歸納出以下觀點：

---

## 二、正面觀點

### 1. **肯定航線擴展努力**
- 網友關注並報導**台北-鳳凰城航線正式開航**，這是星宇在北美的第五個航點
- 對於星宇持續擴展北美航線網絡表示注意，顯示有部分網友認可其發展策略

### 2. **期待機隊發展**
- 網友關注星宇在**新加坡航展展示新機**
- 對**首架廣體客機**的亮相表現出期待
- 顯示部分網友看好星宇未來的硬體升級與長程航線能力

---

## 三、負面/保留觀點

### 1. **競爭力存疑**
- 有網友建議「快去華航吧」，暗示對星宇服務或營運持保留態度
- 提到「長榮華航」作為替代選項，顯示部分乘客仍傾向選擇傳統航空公司

### 2. **機隊規模限制**
- 明確指出「機隊規模小」是星宇的劣勢
- 小機隊可能影響航班頻率、航線穩定性及票價競爭力

### 3. **立場模糊**
- 部分留言如「不是華航」語意不清
- 整體討論內容較片斷，缺乏深入的利弊分析

---

## 四、綜合評估

### 網友態度分布：
- **中立/關注者**：單純報導新航線消息
- **樂觀期待者**：關注新機與硬體升級
- **保守懷疑者**：質疑競爭力，建議選擇傳統航空

### 關鍵爭議點：
1. **機隊規模** vs **服務品質**：小機隊是否能支撐多航點營運？
2. **新興航空** vs **老牌信賴**：星宇能否與華航、長榮競爭？

---

## 五、結論

網友對星宇開拓新航線的反應呈現**「謹慎樂觀但保留懷疑」**的態度。雖然認可其擴張企圖心和硬體投資，但對實際營運能力、服務穩定性仍抱持觀望態度，部分網友更直接表達對傳統航空公司的偏好。星宇需要透過穩定營運與差異化服務來建立消費者信心。
```

Thank you!